

1. Проблематика автоматизации интеллектуального труда.

Эволюция технических средств.

С середины 20-го века человек перешёл в новую эру научно-технической революции и информационного общества. На главную роль вместо физического труда вышел интеллектуальный труд. При этом каждые 15 лет объём информации увеличивался вдвое. В связи с этим, нужно было автоматизировать интеллектуальный труд, так как человек физически не успевал за потоком новой информацией.

Основные проблемы автоматизации интеллектуального труда:

- Огромный рост объёмов информации, необходимой для принятия решений в разных областях;
- Ограниченные возможности человека по обработке информации;
- Многомерность знаний специалистов зафиксированные в образных формах на носителях, при этом одномерность информации на формальных языках старых компьютерных сред.

Вскоре появилось решение проблем хранения и обработки информации – компьютерные среды и первые вычислительные машины.

Автоматизация информационного и интеллектуального трудов были постепенные в различных сферах.

Этапы автоматизации:

- 50-е годы – автоматизация расчётных задач (первые шаги в программирование);
- 60-е годы – автоматизации библиотечной деятельности;
- 70-е годы – автоматизация управленческих задач;
- 80-е годы – автоматизация проектно-конструкторской деятельности и научных исследований;
- 90-е годы – комплексная информатизация различных видов деятельности на основе: телекоммуникационных и мультимедийных информационных технологий.

Автоматизация в перечисленных областях была в виде прикладных автоматизированных систем (ПАС).

Развитие технических средств зависит от: жизненного цикла изделия и

способ производства.

Жизненный цикл – набор производственных процессов от зарождения идеи до утилизации изделия.

Этапы жизненного цикла:

- Осмысление потребности;
- Исследование и анализ осуществимости;
- Проектирование и конструирование;
- Подготовка производства;
- Изготовление;
- Эксплуатация;
- Утилизация.

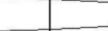
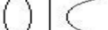
Способы производства:

- **Уникальный** – все этапы жизненного цикла осуществляются одним субъектом;
- **Кустарный** – разделён на процесс создания и процесс использования. Особенность: кустарь не моделирует изделие, изменение формы происходит методом проб и ошибок, форма изделия – хранилище информации.
- **Промышленный** – единичное производство/серийное производство/массовое производство.

Переход к промышленному осуществляется с помощью наличия чертежа.

Особенности:

- Возникает процесс конструирования чертёжным способом;
- Детализовка, масштабирование;
- Разделение труда и специализации;
- Технология изготовления;
- Зависимость различных технологических процессов.

этап ЖЦ способ производства	проектиро вание	подготовка к реализации	изготовление	эксплуатация	утилизация
универсаль ное					
кустарное					
промыш ленное					

2. Проблема создания технических средств.

Основные проблемы:

- Отсутствие программного обеспечения по решению технических проблем;
- Разнообразие видов информации;
- Так как объектом анализа является интеллектуальный труд, то требуется его объективирование;
- Необходимость динамической системы (объёмы информации растут) обработки больших объёмов информации;
- Так как наличие технических средств подразумевает больше промышленный способ производства, то требуется персонал, обладающий знаниями всех этапов жизненного цикла продукта;
- Трудность замены АСУП (автоматизация систем управления предприятием) ERP-системами (информационная система управления бизнес-процессами и ресурсами компании на основе единой базы данных) и АБП (автоматизация бизнес-процессов);
- Новые системы управления предприятием не затрагивают АСУ ТП, этим занимается служба главного метролога.

Схема модификации ТС:

ТС (технические средства) + ПАС (прикладные автоматизированные системы)
= НТС (новые технические средства).

3. Автоматизация этапов жизненного цикла.

Автоматизация жизненных циклов осуществляется при помощи внедрения на производства различных ПАС.

Специфика и общие черты ПАС:

ПАС = ФЧ + ОЧ.

ФЧ (функциональная часть) – отражает структуру функции задачи.

ОЧ (обеспечивающая часть) – комплекс средств автоматизации и виды обеспечения.

Примеры ПАС:

1. АСНИ (автоматизированная система научных исследований);
2. АСНТИ (автоматизированная система научно-технических

исследований);

3. АСТПП (автоматизированная система технологической подготовки производства);
4. АСУ ТП (автоматизированная система управления технологическим процессом) (ЧПУ (числовое программное управление) + ГПС (гибкая производственная система));
5. АСУ (автоматизированная система управления);
6. АИС (автоматическая идентификационная система);
7. ИПС (интегрированная производственная система);
8. БД (база данных);
9. САПР (система автоматизированного проектирования).

ПАС	ФЧ	ОЧ
АСНТИ, ИПС, АИС, БД	Функции ввода, хранение, поиск и вывод информации	1. информационно-методическое 2. организационно-техническое
АСУ	+статистическая обработка данных	(как в какой последовательности)
АСНИ	+сложная вычислительная обработка	3. программно-техническая (с помощью каких средств)
САПР	+ обработка графич. и геометр. информации	Упрощение работы, уменьшение себестоимости.



Обеспечение – это методов и средств для функционирования объекта.

Информационное обеспечение – совокупность единой системы классификаций и массивов информации, использующихся в автоматизированной системе.

Внешнее информационное обеспечение – это методы и средства идентификации объектов, представление их в документах.

Внутреннее информационное обеспечение – это методы и средства преобразования из машинного представления данных во внешнее.

4. Информационные технологии, этапы развития.

Технология – упорядоченная последовательность действий.

Информационная технология – это процесс циркуляции и переработки информации.

Производственная задача состоит из:

- Предмет обработки (информация);
- Процесс обработки (виды процессов по обработке информации);
- Субъект обработки (человек);
- Средства обработки (программно-технические средства).

Виды информационных процессов:

- 1) **Сбор информации** - управление объёмом сведений, который позволяет выполнять поставленные задачи;
- 2) **Передача информации** - перенос информации в пространстве из одной точки в другую;
- 3) **Переработка информации** - целесообразная последовательность действий, направленная на обеспечение принятия решений либо получение результата;
- 4) **Хранение информации** - перенос информации во времени;
- 5) **Вывод информации** - доведение информации до пользователя в удобном для него виде и в удобное для него время.

Технические средства реализации для различных видов информации:

Виды инф. процессов	Виды информации		
	текстовая	образная	звуковая
Сбор, ввод (1)	5,3,13,8 +1	2,5,9 +1	6 +1
Передача (2)	11,12	11,12	11,12
Переработка (3)	10	9,10	10
Хранение (4)	1	1	1
Вывод (5)	2,3,4	2,3,4	7

- 1) Память (ОЗУ, ПЗУ, CD, HD),
- 2) Монитор,
- 3) Принтер,
- 4) Плотер,
- 5) Сканер,
- 6) Микрофон,
- 7) Динамик

8) Манипулятор (мышь джойстик), 9) TV-coder, 10) Процессор,
11) Модем, факс, 12) Телекоммуникации, 13) Клавиатура.

Этапы развития ИТ:

- 1-й этап (1850 года) «ручных» технологий. В качестве средств получения информации применяют перо и чернильницу, книгу, счеты. Коммуникации осуществлялись путём доставки почтой писем, в некоторых странах применялся механический телеграф. Цель этапа – передача информации в необходимом виде.
- 2-й этап (1940 года) «механические» технологии: пишущая машинка, телеграф, телефон, диктофон. Цель этапа – представление информации в нужной форме на более высоком уровне.
- 3-й этап (1960 года) «электрические» технологии: первые ламповые ЭВМ и соответствующее программное обеспечение, электрические пишущие машинки, ксероксы. Цель этапа – доставка и получение информации в заданное время.
- 4-й этап (1970 года) «электронные» технологии: большие ЭВМ и создаваемые на их базе (АСУ). Цель этапа – обработка и рациональное использование информации.
- 5-й этап (1980) «компьютерные» технологии. Появляется персональный компьютер с программными продуктами широкого назначения. Создание систем поддержки принятия решений на различных уровнях управления. Цель этапа – доступность обработки и получение информации для всех.
- 6-й этап (1985 год) обусловлен использованием «Internet» технологий. Широко используются в различных областях деятельности глобальные, региональные и локальные компьютерные сети. Цель этапа – глобализация информации.

5. Электронная обработка данных, эволюция и проблемы.

Электронная обработка данных – это метод обработки данных при помощи систем электронной обработки данных.

Системы электронной обработки данных (СЭОД) – электронная система, которая служат для решения задач, имеющих высокое содержание операций по обработке данных (сбор данных, манипулирование ими, хранение данных и подготовку документов).

Этапы СЭОД:

- 1) Электронная обработка данных;
- 2) Информационные технологии;
- 3) Новые информационные технологии;
- 4) Мультиинформационные среды, средства и технологии.

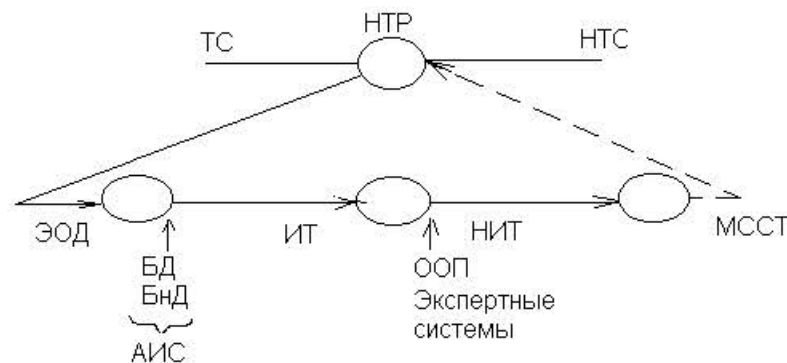
Эволюция ЭОД:

- 1) Программирование кодов, где:
 - **Код** - это адрес физической памяти;
 - **Адрес** – это переменная;
 - **Содержание** – это значение.
- 2) Символическое программирование (вводятся имена переменных);
- 3) Появление ЯПВУ.

Проблемы:

- 1) Рост числа переменных, при решении сложных задач;
- 2) Неявные связи данных в программе;
- 3) Для извлечения данных из файлов необходимо организовать запрос программы;
- 4) Появление аномалий по вводу и обработке данных.

Схема взаимосвязи различных ПАС и методов обработки информации:



ТС (технические средства);

НТР (научно-техническая революция);

НТС (научно-техническая трансформация);

ЭОД (электронная обработка данных);

БНД (банк данных);

ИТ (информационные технологии);

НИТ (новые информационные технологии);

ООП (объектно-ориентированное программирование).

6. Понятие «данные», «информация», «знание».

Данные – это фактическое описание любого явления, которое представляет ценность, или форма представления сведений в процессе их хранения и обработки.

Информация – это приращение знаний, которое может быть выделена на основе данных, отрицательное приращение энтропии.

$$I = -\Delta \mathcal{E}$$

Знание – это целостная систематизированная совокупность научных понятий о закономерностях природы, общества и сознания, накопленные человечеством в процесс активной образовательной деятельности и направленная на дальнейшее познание и преобразование окружающего мира.

Схема взаимосвязи данных, информации и знаний:

Знание - информация – данные.

Общее – специфическое – единичное.

Естественный язык (ЕЯ) - универсальным средством представления данных, информации и знаний.

В вычислительной среде данные хранятся на логических носителях, а их интерпретация возлагается на человека.

Ядром любой информационной системы (ИС) является информация база (БД).

Банк данных (БНД) – это автоматизированная система, обеспечивающая хранение, накопление, поиск и выдачу информации в совокупности программных и технических средств. Состоит из: БД и СУБД.

База данных (БД) - совокупность данных, хранимых в соответствии со схемой данных.

Система управления базами данных (СУБД) - совокупность программных общего или специального назначения, обеспечивающих управление созданием и использованием баз данных.

7. Информационное моделирование. Основные понятия.

Моделирование - исследование объектов познания на их моделях; построение и изучение моделей реально существующих объектов, процессов с целью получения объяснений этих явлений.

Модель – абстракция, позволяющая изучать и исследовать окружающий

мир.

Виды моделей:

- Сетевая;
- Иерархическая;
- Реляционная.

Элементы модели из реального мира:

- 1) Множество предметов окружающей природы;
- 2) Множество связей этих предметов;
- 3) Множество ограничений на элементы и их связи;
- 4) Множество манипуляций с элементами.

Пункты 1-3 – статические, пункт 4 – динамический.

Элементы модели данных:

- 1) Множество элементов данных и их состояний;
- 2) Правила порождения структур данных и их состояний;
- 3) Правила порождения ограничений на структуры данных и их состояния;
- 4) Множество операций над состояниями данных.

Пункты 1-2 – статические, пункты 3-4 – динамические.

Информационное моделирование – это приём научно-технического познания.

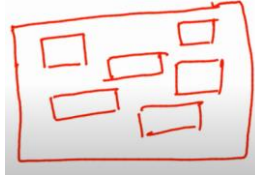
<i>Реальный мир</i>	<i>Информационный мир</i>
Св-ва объектов реального мира	Эл-ты данных (<i>домен</i>)
Объекты (конкретные)	Структура данных (<i>отношение</i>)
Связи объектов	Состояние структур данных (<i>кортеж</i>)
Ограничения на объекты/ на связи объектов	Ограничения на структуры данных и на их состояния
Динамика состояний объектов и их связей	Действия (операции) над состояниями структур

Математический аппарат в основе моделирования данных:

- 1) Формально представлять элементы структур (данные) и ограничения;
- 2) Формально ими манипулировать.

8. Реляционная модель данных. Теоретические основы (математический аппарат, основные понятия).

Реляционная модель данных - совокупность данных, состоящая из набора двумерных таблиц. В теории множеств таблице соответствует термин отношение (relation).



Запись – строка таблицы.

Поля – столбцы таблицы.

Атрибут – поименованный столбец (свойство: рост, имя).

Домен – множество всех возможных значений атрибута.

Кортеж – строка со значениями разных атрибутов (запись).

Математический аппарат реляционных моделей данных – теоретико-множественный подход и алгебра отношений: подмножество декартового произведения списка доменов.

Изначально существует множество два множества вида:

$D = \{d_1, d_2, d_3, \dots, d_n\}$ – множество доменов;

$K = \{k_1, k_2, k_3, \dots, k_n\}$ – множество кортежей.

Тогда декартовое произведение $C = D \times K = \{(d_i, k_j)\}$ – подмножество.

Отношение – подмножество декартового произведения множеств $D \times K$ (фактически, это заполненная таблица, которая не хранит в себе двух одинаков строк).

Арность – размерность отношений (определяется числом доменов).

9. Базовые понятия реляционных баз данных.

Тип данных - предопределённость того, что и как мы будем хранить (int, string, img).

Домен (элемент) – потенциально допустимое множество значений данного типа.

Схема отношений – именованное множество пар {имя атрибута, имя домена}.

Арность – размерность отношений (определяется числом доменов).

Схема БД – это набор именованных схем отношений.

Кортеж (сложное состояние) – множество пар {имя атрибута, значение}, каждое из которых уникальное (набор именованных значений заданного типа).

Арность кортежа = арность схемы отношений.

Отношение – это множество кортежей.

10. Реляционная модель данных (состав модели и элементы).

Состав модели и набор входящих в неё элементов определяется свойствами отношений.

Свойства отношений:

1) Отсутствие кортежей-дубликатов.

У каждого отношения есть **уникальный идентификатор** – набор атрибутов, значения которых определяют кортеж отношений. Следует из определения отношения.

2) Отсутствие упорядоченности кортежей.

Это свойство придаёт гибкость при работе СУБД. Следует из определения отношения.

3) Отсутствие упорядоченности атрибутов.

Следует из определения отношения.

4) Атомарность значений атрибутов.

Значения атрибутов должны быть атомарными. Следует из определения домена.

В реляционных БД в связи с свойствами допускаются только нормализованные отношения.

Нормализованные отношения – отношение в котором, все его атрибуты атомарны.

Нормализация упрощает работу с данными.

Состав модели:

- Структурная часть (представление данных);
- Манипуляционная часть.

В структурной части – фиксируется, что единственной структурой данной в реляционных БД является нормализованное отношение.

В манипуляционной части существуют два механизма манипулирования:

- Реляционная алгебра;
- Реляционное исчисление.

Это абстрактные языки запросов, которые не реализованы в СУБД, но являются эталоном.

Реляционная алгебра базируется на теории множеств. Реляционное исчисление на аппарате исчисления предикатов первого порядка. В свою очередь рассматривают два исчисления:

- Доменов;
- Предикатов.

Все эти механизмы замкнуты относительно понятия отношения. То есть результаты

тоже отношения.

11. Реляционная алгебра. Элементы реляционной алгебры: объединение, разность, декартово произведение, проекция.

Реляционная алгебра - замкнутая система операций над отношениями в реляционной модели данных. Основная идея состоит в том, что если отношения – множество, то манипулирование отношениями могут базироваться на теоретико-множественных операциях, дополненных некоторыми реляционными операциями.

1. **Объединение** (1) $Q = R \cup S$ Объединением отношений R и S называется отношение Q, которое представляет собой множество кортежей, принадлежащих R или S, или им обоим.

Атрибуты из отношения не поименованы, дублирующие кортежи не удваиваются. Основное формальное ограничение – требование одинаковости.

- При выполнении операции объединения двух отношений производится отношение, включающее все кортежи, входящие хотя бы в одно из отношений-операндов.

Пример 1

Отношение A:
Арность отношения A=3
Кол-во кортежей =3

A	B	C
a	b	c
a	a	c
d	b	a

Отношение B:
Арность отношения B=3
Кол-во кортежей =4

D	E	F
a	q	w
a	b	c
a	s	d
f	g	h

Отношение Q:
Кол-во кортежей =6

a	b	c
a	a	c
d	b	a
a	q	w
a	s	d
f	g	h

2. Разность

Разность $Q = R - S$ Разностью двух отношений R и S называется отношение Q , представляющее собой множество кортежей, принадлежащих ($\in$) R и не принадлежащих ($\notin$) S .

Основное формальное ограничение – требование одинаковой арности.

- Отношение, являющееся разностью двух отношений включает все кортежи, входящие в отношение - первый операнд, такие, что ни один из них не входит в отношение, являющееся вторым операндом.

Пример 1

Отношение A:
Арность отношения A=3
Кол-во кортежей =3

A	B	C
a	b	c
a	a	c
d	b	a

Отношение B:
Арность отношения B=3
Кол-во кортежей =4

D	E	F
a	q	w
a	b	c
a	s	d
f	g	h

Отношение Q:
Кол-во кортежей =2

a	a	c
d	b	a

3. Декартово произведение

Пусть отношения R и S обладают арностями k_1 и k_2 соответственно. Тогда декартовым произведением $R \times S$ отношений R и S называется отношение Q , в котором множество кортежей имеют длину $k_1 + k_2$, первые k_1 компонентов которых образуют кортежи, принадлежащие R , а последние k_2 — кортежи, принадлежащие S .

Основное формальное ограничение - арности отношений складываются, а мощности отношений перемножаются.

3. Декартово произведение

$Q = A * B$. Атрибуты результирующего отношения поименованы. Мощности отношений перемножаются. Арности отношений могут быть разными.

Пример 1

Отношение A:

Арность отношения A=3

Кол-во кортежей =3

$$N_A = 3$$

A	B	C	
a	b	c	a1
a	a	c	a2
d	b	a	a3

Отношение B:

Арность отношения B=3

Кол-во кортежей =4

$$N_B = 4$$

D	E	F	
a	q	w	b1
a	b	c	b2
a	s	d	b3
f	g	h	b4

Отношение Q: $N_Q = N_A * N_B = 12$

A	B	C	D	E	F
a1				b1	
a1				b2	
a1				b3	
a1				b4	
a2				b1	
a2				b2	
a2				b3	
a2				b4	
a3				b1	
a3				b2	
a3				b3	
a3				b4	

A	B	C	D	E	F
a	b	c	a	q	w
a	b	c	a	b	c
a	b	c	a	s	d
a	b	c	f	g	h
a	a	c	a	q	w
a	a	c	a	b	c
a	a	c	a	s	d
a	a	c	f	g	h
d	b	a	a	q	w
d	b	a	a	b	c
d	b	a	a	s	d
d	b	a	f	g	h

4. Проекция (всегда понижение арности, 0,)- столбцы

$$Q = \pi_{1...im}(R), im < k1$$

Унарная операция, которая позволяет осуществить доступ к определенной части отношения. Проецирование – это понижение арности.

Идея этой операции заключается в том, что берется отношение R, удаляются некоторые из его компонентов и (или) переупорядочиваются оставшиеся компоненты.

- При выполнении проекции отношения на заданный набор его атрибутов производится отношение, кортежи которого производятся путем взятия соответствующих значений из кортежей отношения-операнда.

Пример 1

Отношение A:

Арность отношения A=3

Кол-во кортежей =3

A	B	C
a	b	c
a	a	c
d	b	a

$\pi_{B,C}(A)$ - Арность результирующего отношения = 2

B	C
b	c
a	c
b	a

12. Реляционная алгебра. Элементы реляционной алгебры: селекция, пересечение, частное, соединение.

Реляционная алгебра - замкнутая система операций над отношениями в реляционной модели данных. Основная идея состоит в том, что если отношения – множество, то манипулирование отношениями могут базироваться на теоретико-множественных операциях, дополненных некоторыми реляционными операциями.

5.Селекция (выбор по критерию) – строки.

$$Q_t = \sigma_k(R)$$

Пусть F — формула, образованная:

- а) операндами, являющимися константами (постоянными) или номерами компонентов;
- б) арифметическими операторами сравнения.

$<, =, >, \leq, \neq, \geq$;

- в) логическими операторами *и*, *или*, *нет*.

Тогда селекция есть множество кортежей t , принадлежащих R , таких, что при подстановки i -го компонента t , вместо любого вхождения i в формулу F для всех i , формула становится истинной.

Как и в проекции, формула в селекции может ссылаться на столбцы по именам, а не по номерам, если столбцы отношения именованы. Заметим также, что константы в формулах должны быть заключены в кавычки. Это позволяет отличать их от номеров или имен столбцов.

Пример 1

Отношение A:

Арность отношения A=3

Кол-во кортежей =3

A	B	C
a	b	c
a	a	c
d	b	a

Селекция $\sigma A = a(A)$ - обозначает множество кортежей, принадлежащих отношению A, первый компонент которых (под именем A) совпадает с символом a.

Кол-во кортежей =2

A	B	C
a	b	c
a	a	c

6. Пересечение $Q = R \cap S = R - (R - S)$

Пересечением отношения R и S является отношение Q, принадлежащее одновременно и R и S.

- Операция пересечения двух отношений производит отношение, включающее все кортежи, входящие в оба отношения-операнда.

$$Q = A \cap B$$

A) abcd

ik mb

cdmg

df ab

B) ik mb

cdmg

cedmf

Q) ik mb

cdmg

7. Частное $Q = R \div S$

Пусть R и S является отношением арности r и s, при этом арность $r > s$, и отношение S- не пустое, тогда частное отношение Q есть множество кортежей t длины r, таких, что для всех кортежей Q

$$Q = A \div B$$

A) abcd

B) cd

Q) ab

ab e f

ef

i k

i k e f

i k c d

abk s

mgnf

8. Соединение

$$Q = R \triangleright \triangleleft S_{iQj}$$

При соединении двух отношений по некоторому условию образуется результирующее отношение, кортежи которого являются конкатенацией кортежей первого и второго отношений и удовлетворяют этому условию.

Q – соединение исходных отношений $i=R$, $j=S$. Есть арифметический оператор сравнения: $>$, $>=$, $=$, $<$, $<=$

$$Q = A \triangleright \triangleleft B$$

A) ABC

B) CD

Q) ABCD

aca

ad

acaad

bba

ba

bbaad

13. Реляционная алгебра: законы операций.

Реляционная алгебра - замкнутая система операций над отношениями в реляционной модели данных. Основная идея состоит в том, что если отношения – множество, то манипулирование отношениями могут базироваться на теоретико-множественных операциях, дополненных некоторыми реляционными операциями.

Законы операций (композиции)

1. коммутативный
2. ассоциативный
3. дистрибутивный

В практике упрощенно называются оптимизацией запроса.

Под отношением мы будем понимать совокупность отображения множества имен атрибутов на множество значений. Два отношения считаются равными, если они считаются одним и тем же множеством отображений.

1. Закон коммутативный для соединения и декартового произведения

$$E1 \times E2 = E2 \times E1$$

$$E1 \triangleright \triangleleft E2 = E2 \triangleright \triangleleft E1$$

$$E1 \triangleright \triangleleft \bowtie E2 = E2 \triangleright \triangleleft \bowtie E1$$

2. Закон ассоциативный для соединения и ДП

$$(E1 \times E2) \times E3 = E1 \times (E2 \times E3)$$

$$(E1 \triangleright \triangleleft E2) \triangleright \triangleleft E3 = E1 \triangleright \triangleleft (E2 \triangleright \triangleleft E3)$$

$$E1 \triangleright \triangleleft \bowtie E2 \triangleright \triangleleft \bowtie E3 = E1 \triangleright \triangleleft \bowtie E2 \triangleright \triangleleft \bowtie E3$$

3. Закон перестановки операции селекция с другими операциями.

$$\sigma_F(E1 - E2) = \sigma_F(E1) - \sigma_F(E2)$$

4. Законы каскадов

$$\sigma_{F1}(\sigma_{F2}(R)) = \sigma_{F1 \wedge F2}(R)$$

Закон каскадов для проекции:

$\pi_{A1...An}(\pi_{B1...Bm}(R)) = \pi_{A1...An}(R)$, справедлив, если множество атрибутов $A1...An \in$ (принадлежит) множеству атрибутов $B1...Bm$

14. Проектирование реляционных БД с использованием нормализации. Определения, проблематика, этапы нормализации.

Проектирование реляционных БД. Технология нормализации

Процесс проектирования представляет собой процесс нормализации схем отношений, причем каждая следующая нормальная форма обладает свойствами лучшими, чем предыдущая. Каждой нормальной форме соответствует некоторый определенный набор ограничений. Отношение находится в некоторой нормальной форме, если удовлетворяет свойственному ей набору ограничений.

Нормализация - метод создания набора отношений с заданными свойствами на основе установленных требований к данным

Нормализация представляет собой процесс проверки и реорганизации отношений и атрибутов с целью удовлетворения требованиям к реляционной модели данных. В результате проведения нормализации должна быть создана структура данных, при которой информация о каждом факте хранится только в одном месте.

Процесс нормализации сводится к последовательному приведению структуры данных к нормальным формам - формализованным требованиям к организации данных.

Примером набора ограничений является ограничение первой нормальной формы - значения всех атрибутов отношения атомарны.

Требование первой нормальной формы является базовым требованием классической реляционной модели данных.

Выделяется следующая последовательность нормальных форм:

Основные свойства нормальных форм:

- каждая следующая нормальная форма в некотором смысле лучше предыдущей;
- при переходе к следующей нормальной форме свойства предыдущих нормальных свойств сохраняются.

В основе процесса проектирования лежит метод нормализации, декомпозиция отношения, находящегося в предыдущей нормальной форме, в два или более отношения, удовлетворяющих требованиям следующей нормальной формы.

Выделяется следующая последовательность нормальных форм:

- первая нормальная форма (1NF);
- вторая нормальная форма (2NF);
- третья нормальная форма (3NF);
- нормальная форма Бойса-Кодда (BCNF);

- четвертая нормальная форма (4NF);
- пятая нормальная форма, или нормальная форма проекции-соединения (5NF или PJ/NF).

15. Проектирование реляционных БД с использованием нормализации (1НФ, 2НФ, 3НФ).

Проектирование реляционных БД. Технология нормализации

Процесс проектирования представляет собой процесс нормализации схем отношений, причем каждая следующая нормальная форма обладает свойствами лучшими, чем предыдущая. Каждой нормальной форме соответствует некоторый определенный набор ограничений. Отношение находится в некоторой нормальной форме, если удовлетворяет свойственному ей набору ограничений.

Нормализация - метод создания набора отношений с заданными свойствами на основе установленных требований к данным

Нормализация представляет собой процесс проверки и реорганизации отношений и атрибутов с целью удовлетворения требованиям к реляционной модели данных. В результате проведения нормализации должна быть создана структура данных, при которой информация о каждом факте хранится только в одном месте.

Процесс нормализации сводится к последовательному приведению структуры данных к нормальным формам - формализованным требованиям к организации данных.

Примером набора ограничений является ограничение первой нормальной формы - значения всех атрибутов отношения атомарны.

Требование первой нормальной формы является базовым требованием классической реляционной модели данных.

Выделяется следующая последовательность нормальных форм:

Основные свойства нормальных форм:

- каждая следующая нормальная форма в некотором смысле лучше предыдущей;
- при переходе к следующей нормальной форме свойства предыдущих нормальных свойств сохраняются.

В основе процесса проектирования лежит метод нормализации, декомпозиция отношения, находящегося в предыдущей нормальной форме, в два или более отношения, удовлетворяющих требованиям следующей нормальной формы.

Одним из атрибутов отношения должен однозначно определять отдельный кортеж (т.е. по его значению можно найти один и только один кортеж). Такой атрибут называется **ключевым** или **первичным ключом** отношения. Иногда ключ может состоять из нескольких атрибутов, тогда он называется **составным**. В отношении не должно быть двух и более кортежей с одинаковыми значениями ключа. Это свойство называется **целостностью сущности**. *Ключ отношения можно определить как совокупность атрибутов, от которой функционально зависят все остальные атрибуты отношения. При этом в составной ключ должны входить только функционально независимые атрибуты.*

Нормализация – это пошаговый обратимый процесс разложения исходных отношений на более мелкие и простые.

Определение: Отношение находится в *первой нормальной форме* (1НФ), если все его атрибуты являются атомарными и соблюдается целостность сущности, т.е. имеется первичный ключ отношения.

Для приведения к 1НФ нужно:

1. Заполнить таблицы БД, максимально дублируя строки;
2. Избавиться от сложных атрибутов;
3. Определить ключ каждого отношения.

Определение: Отношение находится во 2НФ, если оно находится в 1НФ и каждый неключевой атрибут функционально полно зависит от составного ключа.

Если отношение находится в 1НФ, то все неключевые атрибуты функционально зависят от ключа. Но степень формализации может быть различной.

Частичная зависимость – если неключевой атрибут зависит от части ключа.

Полная функциональная зависимость – когда неключевой атрибут зависит от всего ключа целиком.

В нашем примере **Название** зависит только от **Кода ВУЗа**, т.е. налицо частичная зависимость. Полная зависимость наблюдается между ключом **Код ВУЗа+Предмет** и атрибутом **Часы**.

Для этого необходимо:

4. указать письменно частичные зависимости неключевых атрибутов от ключа;
5. разложить исходное отношение на два:
 - построить отношение, в которое входит часть составного ключа и атрибуты, зависящие только от этой части. Повторяющиеся кортежи нужно удалить;
 - построить отношение, в которое входят полный ключ и оставшиеся атрибуты.
6. Указать ключи в каждом из получившихся отношений и установить от первого (главного – со стороны *один*) отношения ко второму (подчиненному – со стороны *многие*) связи **1:М** по внешнему ключу, которым является часть составного ключа во втором отношении.
7. *Еще раз рассмотреть каждое из отношений БД на наличие частичных зависимостей и проделать заново пункты 4-6, если такие найдены.*

Определение: **Внешним ключом** является один из неключевых атрибутов отношения, который в другом отношении является первичным ключом.

Определение: **Ссылочной целостностью** называется такое состояние Базы данных, при котором каждому значению внешнего ключа подчиненного отношения соответствует кортеж другого (главного) отношения с таким же значением первичного ключа. Обратного соответствия не требуется.

Определение: Отношение находится в 3НФ, если оно находится во 2НФ и в нем отсутствуют транзитивные зависимости неключевых атрибутов от ключа.

В полученном отношении **ВУЗЫ** имеются следующие аномалии:

- Дублированные информации о регионе для ВУЗов одного города;
- Проблема контроля из-за избыточности данных, поскольку изменение названия региона, в котором находится город, влечет за собой необходимость поиска и изменения этого названия у всех ВУЗов этого города;
- Нельзя включить данные о новом городе, если на данный момент в нем еще отсутствуют ВУЗы и, наоборот, при упразднении всех ВУЗов в городе данные о нем нельзя сохранить.

Говорят, что атрибут C зависит от атрибута A транзитивно, если есть атрибут-посредник B такой, для которого выполняются зависимости $A \rightarrow B$ и $B \rightarrow C$, и при этом обратные зависимости отсутствуют. Такую зависимость можно изобразить следующим образом: $A \rightarrow B \rightarrow C$.

Необходимо удалить все транзитивные зависимости неключевых атрибутов от ключа:

8. Рассмотреть каждую из таблиц на наличие транзитивных зависимостей неключевых атрибутов от ключа. Указать письменно такие зависимости.
9. Если транзитивных зависимостей нет, доказать это письменно;
10. Разложить исходное отношение на два:
 - построить отношение, в которое входит зависимый атрибут и атрибут-посредник. Повторяющиеся кортежи нужно удалить;
 - построить отношение, в которое входят все атрибуты, кроме зависимого;
11. Указать ключи в каждом из получившихся отношений и установить от первого (главного) отношения ко второму (подчиненному) связь 1:М по внешнему ключу, которым является посредник во втором отношении.
12. Еще раз рассмотреть каждое из отношений на наличие транзитивных зависимостей и проделать заново пункты 8-11, если такие найдены.

16. Проектирование реляционных БД с использованием нормализации (Нормальная форма Бойса-Кодда, 5НФ, 6НФ)

Проектирование реляционных БД. Технология нормализации

Процесс проектирования представляет собой процесс нормализации схем отношений, причем каждая следующая нормальная форма обладает свойствами лучшими, чем предыдущая. Каждой нормальной форме соответствует некоторый определенный набор ограничений. Отношение находится в некоторой нормальной форме, если удовлетворяет свойственному ей набору ограничений.

Нормализация - метод создания набора отношений с заданными свойствами на основе установленных требований к данным

Нормализация представляет собой процесс проверки и реорганизации отношений и атрибутов с целью удовлетворения требованиям к реляционной модели данных. В результате проведения нормализации должна быть создана структура данных, при которой информация о каждом факте хранится только в одном месте.

Процесс нормализации сводится к последовательному приведению структуры данных к нормальным формам - формализованным требованиям к организации данных.

Примером набора ограничений является ограничение первой нормальной формы - значения всех атрибутов отношения атомарны.

Требование первой нормальной формы является базовым требованием классической реляционной модели данных.

Выделяется следующая последовательность нормальных форм:

Основные свойства нормальных форм:

- каждая следующая нормальная форма в некотором смысле лучше предыдущей;
- при переходе к следующей нормальной форме свойства предыдущих нормальных свойств сохраняются.

В основе процесса проектирования лежит метод нормализации, декомпозиция отношения, находящегося в предыдущей нормальной форме, в два или более отношения, удовлетворяющих требованиям следующей нормальной формы.

Нормальная форма Бойса-Кодда (НФБК) (частная форма третьей нормальной формы)

Определение 3НФ не совсем подходит для следующих отношений:

- 1) отношение имеет два или более потенциальных ключа;
- 2) два и более потенциальных ключа являются составными;
- 3) они пересекаются, т.е. имеют хотя бы один общий атрибут.

Для отношений, имеющих один потенциальный ключ (первичный), НФБК является 3НФ.

Отношение находится в НФБК, когда каждая нетривиальная и неприводимая слева функциональная зависимость обладает потенциальным ключом в качестве детерминанта.

Отношение находится в 4НФ, если оно находится в НФБК и все нетривиальные многозначные зависимости фактически являются функциональными зависимостями от ее потенциальных ключей.

В отношении $R(A, B, C)$ существует **многозначная зависимость** $R.A \twoheadrightarrow R.B$ в том и только в том случае, если множество значений B , соответствующее паре значений A и C , зависит только от A и не зависит от C .

То есть, например, при добавлении нового вида пиццы придется внести по одному новому кортежу для каждого района доставки. Возможна логическая аномалия, при которой определенному виду пиццы будут соответствовать лишь некоторые районы доставки из обслуживаемых рестораном районов.

Для предотвращения аномалии нужно декомпозировать отношение, разместив независимые факты в разных отношениях. В данном примере следует выполнить декомпозицию на {Ресторан, Вид пиццы} и {Ресторан, Район доставки}.

Однако, если к исходной переменной отношения добавить атрибут, функционально зависящий от потенциального ключа, например цену с учётом стоимости доставки ($\{Ресторан, Вид пиццы, Район доставки\} \rightarrow Цена$), то полученное отношение будет находиться в 4НФ и его уже нельзя подвергнуть декомпозиции без потерь.

Отношения находятся в 5НФ, если оно находится в 4НФ и отсутствуют сложные зависимые соединения между атрибутами.

Если «Атрибут_1» зависит от «Атрибута_2», а «Атрибут_2» в свою очередь зависит от «Атрибута_3», а «Атрибут_3» зависит от «Атрибута_1», то все три атрибута обязательно входят в один кортеж.

Пятая нормальная форма ориентирована на работу с зависимыми соединениями. Указанные зависимые соединения между тремя атрибутами встречаются очень редко. Зависимые соединения между четырьмя, пятью и более атрибутами указать практически невозможно.

Переменная отношения находится в шестой нормальной форме тогда и только тогда, когда она удовлетворяет всем нетривиальным зависимостям соединения. Из определения следует, что переменная находится в 6НФ тогда и только тогда, когда она неприводима, то есть не может быть подвергнута дальнейшей декомпозиции без потерь. Каждая переменная отношения, которая находится в 6НФ, также находится и в 5НФ.

Идея «декомпозиции до конца» выдвигалась до начала исследований в области хронологических данных, но не нашла поддержки. Однако для хронологических баз данных максимально возможная декомпозиция позволяет бороться с избыточностью и упрощает поддержание целостности базы данных.

17. Семантические модели данных.

Семантические модели данных

Потребности проектировщиков баз данных в более удобных средствах моделирования предметной области возродили направление семантических моделей данных. При том, что любая развитая семантическая модель данных, как и реляционная модель, включает структурную, манипуляционную и целостную части, главным назначением семантических моделей является обеспечение возможности выражения семантики данных.

Наиболее часто на практике семантическое моделирование используется на первой стадии проектирования базы данных.

Этот процесс выполняется под управлением методик, в которых четко оговорены все этапы такого преобразования.

ER - модель

На использовании разновидностей ER-модели основано большинство современных подходов к проектированию баз данных (главным образом, реляционных). Модель была предложена Ченом в 1976 г.

Подход дает возможность многообразия данного мира представлять универсально. В основе его лежит деление реального мира на отдельные сущности, которые находятся в определенной связи друг с другом.

А атрибут – это свойство сущности.

18. Основные понятия ER- модели.

Основные понятия модели Entity-Relationship (Сущность-Связи)

Моделирование предметной области базируется на использовании графических диаграмм, включающих небольшое число разнородных компонентов. В связи с наглядностью представления концептуальных схем баз данных ER-модели получили широкое распространение в системах CASE, поддерживающих автоматизированное проектирование реляционных баз данных.

Основными понятиями ER-модели являются **сущность, связь и атрибут**.

Сущность - это реальный или представляемый объект, информация о котором должна сохраняться и быть доступна. В диаграммах ER-модели сущность представляется в виде прямоугольника, содержащего имя сущности. При этом имя сущности - это имя типа, а не некоторого конкретного экземпляра этого типа.

Каждый **экземпляр сущности** должен быть отличим от любого другого экземпляра той же сущности.

Связь - это графически изображаемая ассоциация, устанавливаемая между двумя сущностями. Эта ассоциация всегда является бинарной и может существовать между двумя разными сущностями или между сущностью и ей же самой (рекурсивная связь). В любой связи выделяются два конца (в соответствии с существующей парой связываемых сущностей), на каждом из которых указывается имя конца связи, степень конца связи (сколько экземпляров данной сущности связывается), обязательность связи (т.е. любой ли экземпляр данной сущности должен участвовать в данной связи).

Атрибутом сущности является любая деталь, которая служит для уточнения, идентификации, классификации, числовой характеристики или выражения состояния сущности. Имена атрибутов заносятся в прямоугольник, изображающий сущность, под именем сущности и изображаются малыми буквами, возможно, с примерами.

19. Получение реляционной схемы из ER-схемы.

Получение реляционной схемы из ER-схемы

- Каждая простая сущность превращается в таблицу. Простая сущность - сущность, не являющаяся подтипом и не имеющая подтипов. Имя сущности становится именем таблицы.
- Каждый атрибут становится возможным столбцом с тем же именем;
- Компоненты уникального идентификатора сущности превращаются в первичный ключ таблицы. Если имеется несколько возможных уникальных идентификатора, выбирается наиболее используемый.
- Связи многие-к-одному (и один-к-одному) становятся внешними ключами.
- Индексы создаются для первичного ключа (уникальный индекс), внешних ключей и тех атрибутов, на которых предполагается в основном базировать запросы.

20. Ограничение целостности модели данных (Основные понятия, виды целостности и целостность по ссылкам).

Ограничение целостности модели данных

Целостность (от англ. integrity – нетронутость, неприкосновенность, сохранность)

Целостность – это состояние данных, когда они сохраняют свое информационное содержание и однозначность интерпретации в условиях случайных воздействий.

Но это может быть достигнуто лишь в определенных пределах: СУБД не может контролировать правильность каждого отдельного значения, вводимого в базу данных (хотя каждое значение можно проверить на правдоподобность).

Целостность данных считается сохраненной, если данные не искажены и не разрушены.

Ограничение целостности – это правила, определяющие логические ограничения, накладываемые на данные, используемые СУБД для поддержания целостности данных.

Источниками ошибок при нарушении целостности данных могут быть:

1. Преднамеренное разрушение данных - проблема безопасности
2. случайная ситуация, непреднамеренная (случайное разрушение – случайно что-нибудь стереть.) или полученное в результате вычисления значений, не соответствующих характеристикам реального объекта.

Основные виды целостности

Современные СУБД имеют ряд средств для обеспечения поддержания целостности (так же, как и средств обеспечения поддержания безопасности).

Выделяют три группы правил целостности:

1. Целостность по сущностям.
2. Целостность по ссылкам.
3. Целостность, определяемая пользователем.

Целостность ссылок Значение внешнего ключа должно либо:

- быть равным значению первичного ключа цели;
- быть полностью неопределенным, т.е. каждое значение атрибута, участвующего во внешнем ключе должно быть неопределенным.

Пусть, например, даны отношения ОТДЕЛ (N_ОТДЕЛА, ИМЯ_ОТДЕЛА) и СОТРУДНИК (N_СОТРУДНИКА, N_ОТДЕЛА, ИМЯ_СОТРУДНИКА), в которых хранятся сведения о работниках предприятия и подразделениях, где они работают. Отношение ОТДЕЛ в данной паре является родительским, поэтому его первичный ключ "N_отдела" присутствует в дочернем отношении СОТРУДНИК. Требование целостности по ссылкам означает здесь, что в таблице СОТРУДНИК не может присутствовать кортеж со значением атрибута "N_отдела", которое не встречается в таблице ОТДЕЛ. Если такое значение в отношении ОТДЕЛ отсутствует, значение внешнего ключа в отношении СОТРУДНИК считается неопределенным.

Как правило, поддержание целостности ссылок также возлагается на систему управления базой данных. Например, она может не позволить пользователю добавить запись, содержащую внешний ключ с несуществующим (неопределенным) значением.

21. Ограничение целостности модели данных (Основные понятия, виды целостности и целостность, определяемая пользователем).

Ограничение целостности модели данных

Целостность (от англ. integrity – нетронутость, неприкосновенность, сохранность)

Целостность – это состояние данных, когда они сохраняют свое информационное содержание и однозначность интерпретации в условиях случайных воздействий.

Но это может быть достигнуто лишь в определенных пределах: СУБД не может контролировать правильность каждого отдельного значения, вводимого в базу данных (хотя каждое значение можно проверить на правдоподобность).

Целостность данных считается сохраненной, если данные не искажены и не разрушены.

Ограничение целостности – это правила, определяющие логические ограничения, накладываемые на данные, используемые СУБД для поддержания целостности данных.

Источниками ошибок при нарушении целостности данных могут быть:

1. Преднамеренное разрушение данных - проблема безопасности
2. случайная ситуация, непреднамеренная (случайное разрушение – случайно что-нибудь стереть.) или полученное в результате вычисления значений, не соответствующих характеристикам реального объекта.

Основные виды целостности

Современные СУБД имеют ряд средств для обеспечения поддержания целостности (так же, как и средств обеспечения поддержания безопасности).

Выделяют три группы правил целостности:

1. Целостность по сущностям.
2. Целостность по ссылкам.
3. Целостность, определяемая пользователем.

Целостность определяемая пользователем. Для любой конкретной базы данных существует ряд дополнительных специфических правил, которые относятся к ней одной и определяются разработчиком. Чаще всего контролируется: уникальность тех или иных атрибутов, диапазон значений экзаменационная оценка от 2 до 5), принадлежность набору значений (пол "М" или "Ж").

В общем случае целостность данных может нарушаться по следующим основным причинам:

- 1) ошибки в создании структуры локальных БД и их заполнении;
- 2) просчеты в построении структуры РБД (процедуры фрагментации и локализации);
- 3) системные ошибки в программном обеспечении взаимодействия локальных БД (одновременный доступ);
- 4) аварийная ситуация (неисправность технических средств) и восстановление РБД.

22. Реляционная модель данных (достоинства и недостатки).

Достоинства и недостатки реляционного подхода

Реляционная модель данных наряду с общепризнанными достоинствами обладает и рядом недостатков. К числу достоинств реляционного подхода можно отнести:

- наличие небольшого набора абстракций, которые позволяют сравнительно просто моделировать большую часть распространенных предметных областей и допускают точные формальные определения, оставаясь интуитивно понятными;
- наличие простого и в то же время мощного математического аппарата, опирающегося главным образом на теорию множеств и математическую логику и обеспечивающего теоретический базис реляционного подхода к организации баз данных;

Недостатки реляционных СУБД:

- некоторая ограниченность (прямое следствие простоты) при использовании в так называемых нетрадиционных областях (наиболее распространенными примерами являются САПР), в которых требуются предельно сложные структуры данных.
- невозможность адекватного отражения семантики предметной области (возможности представления знаний о семантической специфике предметной области в реляционных системах очень ограничены).

23. Сетевая модель данных.

Формальный аппарат представлен в виде теории графов.

$G=(A,a)$,

Где А-множество вершин (множество логических записей), а- множество дуг (множество связей).

Все логические записи разбиваются на классы и типы.

$$A = \bigcup_{i=1}^n A_i, A_i \cap A_j = \emptyset \text{ (пересечение)}$$

A_i определяет класс или тип логической записи, состоит из множества логических записей.

$A_i = \{a_{in}\}$, a_{in} принадлежит A_i , где a_{in} – n-ая логическая запись i-го типа.

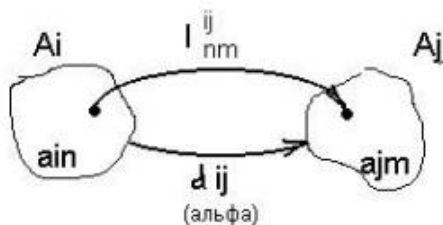
$A_j = \{a_{jm}\}$, a_{jm} принадлежит A_j , где a_{jm} – m-ая логическая запись j-го типа.

Множество связей α – это объединение типов i и j.

$$\alpha = \bigcup \alpha_{ij}$$

$$l_{nm}^{ij} \in \alpha_{ij}, \alpha_{ij} = \{l_{nm}^{ij}\}$$

Тип логической записи по смыслу эквивалентен шапке отношения.



Сетевые системы

Сетевой подход к организации данных является расширением иерархического.

Сеть, которая используется в сетевой модели – граф. У вершины может быть больше одного предка (родителя), граф может быть ориентирован.

В иерархических структурах запись-потомок должна иметь в точности одного предка; в сетевой структуре данных потомок может иметь любое число предков.

Сетевая БД состоит из набора записей и набора связей между этими записями.

Тип связи определяется для двух типов записи: предка и потомка. Экземпляр типа связи состоит из одного экземпляра типа записи предка и упорядоченного набора экземпляров типа записи потомка.

Заключение

Достоинства:

- Развитые средства управления данными во внешней памяти на низком уровне;
- Возможность построения эффективных прикладных систем;

Недостатки:

- Слишком сложные в использовании;
- Фактически необходимы знания о физической организации, следствие - прикладные системы зависят от этой организации;
- Сложности в организации доступа к БД
- Отсутствие аппарата манипуляций над структурами, следствие – ограниченное управление структурами данных

24. Иерархическая модель данных.

Иерархические системы

Первая версия появилась в 1968 г.

Иерархия – это сеть, являющаяся лесом, т.е. совокупностью деревьев. Вводится понятие виртуальной логической записи – в отличие от сетевой модели такая запись представляет собой указатель на логическую запись некоторого типа.

Иерархическая БД состоит из упорядоченного набора деревьев; более точно, из упорядоченного набора нескольких экземпляров одного типа дерева. Тип дерева состоит из одного "корневого" типа записи и упорядоченного набора типов поддеревьев (каждое из которых является некоторым типом дерева). Тип дерева в целом представляет собой иерархически организованный набор типов записи.

25. Сравнение сетевой и иерархической моделей данных.

Соотношение понятий в моделях данных

Реальный мир	Информационный мир		
	Абстрактная МД	Реляц МД	Сетевая МД Иерарх МД
Свойства объектов	Элемент данных	Домен	Поле логич записи
объект	Структура	Отношение	Тип логич записи
Ограничения на объекты/ на связи объектов	Ограничения на структуры данных и на их состояния	Кортеж	Набор/связь
Динамика состояний объектов и их связей	Действия (операции) над состояниями структур	Реляц алгебра и реляц исчисление на основе теории множеств	-----

Заключение

Достоинства:

- Развитые средства управления данными во внешней памяти на низком уровне;
- Возможность построения эффективных прикладных систем;

Недостатки:

- Слишком сложные в использовании;
- Фактически необходимы знания о физической организации, следствие - прикладные системы зависят от этой организации;
- Сложности в организации доступа к БД
- Отсутствие аппарата манипуляций над структурами, следствие – ограниченное управление структурами данных

26. Даталогическое моделирование.

1. Даталогическое моделирование

Даталогическая модель (ДЛМ) - Допустимое представление данных с помощью определенных программно-технических средств т.е. ДЛМ д.б. всегда ориентирована на среду реализации. (программно-техническую среду (ПТСр))

Из этого следует, что большое разнообразие программно-технических средств предполагает разнообразие допустимых даталогических моделей.

Под программно-технической средой (ПТСр) будем понимать совокупность следующих факторов:

- Типохарактеристика информационного пространства
- Типохарактеристика вычислительной техники (ЭВМ+периферийные устройства)
- Типохарактеристика операционной системы. Пакет программ, предназначенный для выполнения определенных функций по обработке различных видов информации, встраиваемый в определенную программно-техническую среду.

Программно-технические средства (ПТС) – это пакет \ комплекс программ, предназначенный для выполнения специфичных функций (обработка структурированных данных, текстов, графической информации и др.) и встраиваемый в определенную программно-техническую среду. Это специализированные средства для выполнения каких-либо функций по обработке информации.

СУБД - это совокупность языковых и программных средств предназначенных для создания, ведения и использования БД многими пользователями.

СУБД объединяет 2 идеи:

1. Идея структуризации данных
2. Централизация хранения информации и децентрализация использования ее.

Каждая СУБД поддерживает определенное модельное представление информации, которое называется дано-логическое представление СУБД.

После этапа выбора программно-технических средств следует создание даталогической модели предметной задачи, ориентированной на выбранную СУБД.

Под ДЛМ предметной задачи понимают совокупность структур данных и алгоритмов программ на эти структуры, обеспечивающих автоматизированное решение предметной задачи (т.е. это планирование организации решения нашей предметной задачи в определенной Программно-Технической среде с помощью определенных ПТ средств. (ДЛМПЗ))

Организация решения - это организация представления данных и организация технических процессов.

27. Структура ДЛМ.

ДЛМ 2 уровня:

- Логический
- Физический

ДЛМ Логический уровень:

Обобщенный уровень – концептуальная модель

Локальный уровень – внешняя модель

Физический уровень:

Обобщенный уровень – внутренняя модель

Внешняя модель: В архитектуре СУБД модель данных на внешнем уровне отображает представление пользователя о БД.

Концептуальная модель – Информационная модель предметной области в терминах конкретной СУБД, содержащая полный набор данных и связей.

Внутренняя модель – в архитектуре СУБД слой нижнего физического уровня отображающий представление данных во внешней памяти и методы доступа к нему.

Так как ДЛМ должна отображать описание данных и манипулирование ими, то язык ДЛМ разделяется на ЯОД (язык описания данных) и ЯМД (язык манипулирования данными).

28. Языки ДЛМ.

Программные средства СУБД

Программные средства СУБД включают в себя набор программных модулей, системных библиотек и определенную структуру файлов, для обеспечения выполнения основных функций по вводу, хранению, поиску и выдаче информации при различных вариантах организации интерфейса с пользователем.

Как правило каждому действию соответствует программный модуль (процедура). Кроме того дополнительные программные модули обеспечивают функцию установки СУБД и инициализации пользователей, реорганизации и восстановления в случае возникновения различных аварийных ситуаций.

29. Задача выбора программно-технических средств для реализации автоматизированной информационной системы.

Постановка задачи фактически аналогична задаче выбора технологического оборудования для изготовления детали по чертежу.

Исходная информация для решения задачи выбора ПТС включает следующее описание:

*

- 1) информационная модель представляет собой совокупность информационной структуры (то есть определенной организации данных) и алгоритм решения задачи на этой структуре (то есть действия по обработке, доступу и фиксации данных);
- 2) ограничения или условия, связанные с эксплуатацией автоматизированной системы;
- 3) документация по различным СУБД (описание характеристики ПТ средств).

Цель задачи: необходимо выбрать наилучшую СУБД в соответствии с критерием: минимальные суммарные затраты на разработку, реализацию и эксплуатацию АИС (автоматизированной информационной системы) для нашей предметной задачи с помощью выбранной СУБД. При этом выбранное ПТ средство должно полностью реализовать все функции, которые отражены в алгоритме.

$$\min(\sum (z_{real}^{AIS} + z_{ekspluat}^{AIS})), \text{ где } z - \text{ это затраты.}$$

Особенности решения задачи:

1. неполнота исходной информации
2. многоаспектность сравниваемых альтернатив.

Метод решения -задачи выбора ПТС (программно-технических средств) может включать в себя несколько итераций, каждая из которых состоит из двух этапов: качественный (1) и количественный (2) анализ.

- 1) Суть качественного этапа заключается в определении качественного соответствия наших требований и существующих возможностей. (т.е. соответствия между информационной моделью предметной задачи и ограничениями на ее эксплуатацию с одной стороны и возможностями различных СУБД по их реализации с другой стороны.)
- 2) Суть количественного анализа заключается в определении степени этого соответствия.

На этапе количественного анализа формируется шкала (диапазон) параметров для описаний предметной задачи и проводится оценка степени соответствия требований предметной задачи возможностям отобранных СУБД, то есть проводят многокритериальную оценку пригодности СУБД (на основе свертки критериев).

30. Схема параметризации исходной информации задачи.

При описании программно-технических средств выделяют 3 уровня:

1. Внешний по отношению к этому ПТСр-ву

К этому уровню относятся: класс вычислительной техники, операционные среды, взаимодействие с периферийным оборудованием и настройка.

Группа параметров: внешние параметры.

2. Описание содержимого ПТСр

К 2-му относится: Виды интерфейса для выполнения основных функций по обработке информации, спектр дополнительных функций для обработки информации.

Группа параметров: внутренние функциональные параметры (функции, составляющие алгоритм «обработки проекта»)

3. Внутренний – содержание ПТСр

Относится: Описание структуры и особенности ДИМ, поддерживаемой данными ПРСр-ми, (состав структурных единиц для хранения, представления, обработки информации, количественная оценка структурных единиц и связей между ними).

Группа параметров: Внутренние структурные параметры (форма, отчет и т.д.)

31. Организация интерфейса пользователя СУБД.

Как правило, интерфейс СУБД делится на 2 вида:

- программный
- диалоговый.

Интерфейс

Диалоговый (чистый ЯМД)	программный	
	Процедурный	Программный
Позволяет: - манипулировать любыми структурными единицами - вспомогательные компоненты - информационно-справочный блок - системы команд	ЯМД→ЯП - транслятор типа интерпретатор/ Транслирует и выполняет построчно. - dBase	В ЯП вставлены команды ЯМД - компилятор - CAL-интерфейс - пошаговая компиляция

Особенности CAL- интерфейса:

1. В тексте программы на определенном ЯП фиксируется обращение к функциям и процедурам, которые являются компонентами СУБД.
2. Обмен данными между программой и СУБД – это замещение формальных параметров фактическими.
3. При компиляции такой программы необходимо подключать соответствующую библиотеку.

32. Критерии реляционности СУБД.

Существует понятие «уровень реляционности» СУБД.

Автор реляционной модели Кодд сформулировал 333 правила.

По этим правилам БД может квалифицироваться как достаточно реляционная и полностью реляционная.

В настоящее время не существует коммерческой полностью реляционной СУБД.

Сейчас существуют вполне реляционные СУБД. Например Ingress, Paradox и т.д.

Требования по Кодду:

1. Реляционная БД должна обеспечивать доступ к данным на логическом уровне, а не на физическом (запрос SQL).
2. Выбор данных должен происходить по именам и значениям без явного перемещения по записям.
3. Не пользователь-программист, а система должна выбирать наилучший способ выполнения запроса (непроцедурный метод доступа к данным).
4. полный набор реляционных операций должен происходить как над исходными записями, так и над совокупностью записей в результате запроса.

33. Структура СУБД Access, соотношение средств для логической и физической реализации.

Языковые средства СУБД включают набор структурных элементов и отношения между ними.

Средства для логической организации данных подразделяются на:

- 1) средства для хранения информации (структурные элементы - база данных, таблица, столбец);
- 2) средства для представления информации (структурные элементы - форма, отчет, меню);
- 3) средства для обработки информации (структурные элементы – макросы, запросы SQL и QBE, язык VBA: оператор / команда, программа, программный модуль).

Средства для физической организации данных включают структурные элементы: файл, сегмент (компонент) файла, записи и поля файла.

В зависимости от способа организации различают диалоговый и программный интерфейс пользователей с СУБД:

1. Язык для организации диалогового интерфейса является базовым и включает действия по манипулированию структурными элементами и сервисные действия.

(В системе Access интерактивная часть (или диалоговый интерфейс) включает в себя иерархическую систему меню для создания таблиц, ввода данных, просмотра и редактирования содержимого таблиц, выполнения запросов, создания отчетов, форм, графиков, написания скриптов.);

2. При программном интерфейсе программный модуль создается на языке программирования высокого уровня Visual Basic, также существует библиотека, с помощью которой можно получить доступ к таблицам СУБД. Это вариант с включающим (базовым) языком.

34. Архитектура MS Access.

Microsoft Access называет *объектами* все, что может иметь имя (в смысле Access). В базе данных Access основными объектами являются таблицы, запросы, формы, отчеты, макросы и модули.

Если вы на своем компьютере использовали другие СУБД, то, возможно, заметили, что термин *база данных* обычно относился только к файлам, в которых вы хранили данные. В Microsoft Access база данных включает в себя все объекты, связанные с хранимыми данными, в том числе и те, которые вы определяете для автоматизации работы с ними. Ниже приведен список основных объектов базы данных Access.

Таблица Объект, используемый для хранения данных. Каждая таблица включает информацию об объекте определенного типа. Таблица содержит *поля (столбцы)*, в которых хранятся различного рода данные, например фамилия или адрес клиента, и *записи* (которые называются также *строками*).

Запрос Объект, который позволяет пользователю получить нужные данные из одной или нескольких таблиц. Для создания запроса можно использовать бланк QBE (запрос по образцу) или инструкции SQL. Существуют запросы на выборку, обновление, удаление или добавление данных.

Форма Объект, предназначенный в основном для ввода данных, отображения их на экране или управления работой приложения. Формы используют для реализации требований пользователя к представлению данных из запросов или таблиц.

Отчет Объект, предназначенный для создания документа, который впоследствии может быть распечатан или включен в документ другого приложения.

Макрос Объект, представляющий собой структурированное описание одного или нескольких действий, которые, должен выполнить Access в ответ на определенное событие.

Модуль Объект, содержащий программы, написанные на языке Visual Basic для приложений, которые позволяют вам разбить некоторый процесс на несколько процедур.

35. Типы данных Microsoft Access и их представление.

Всего поддерживается 10 основных типов данных.

- Текстовый. Принят в СУБД Access по умолчанию для новых полей. Сюда можно поместить любые алфавитно-цифровые символы, включая числа.
- Поле МЕМО. Текстовое поле очень большой длины.
- Числовой. СУБД Access позволяет выполнять над полями такого типа вычисления.
- Дата/время. Спец поле, в котором хранятся значения даты и времени. Сохраняет 8 байт.
- Счетчик. Уникальный идентификатор, основная функция — нумеровать каждую новую запись.
- Логический. Такие поля содержат значения логического типа: Да/Нет, Истина/Ложь, Вкл/Выкл и т.п.
- Поле объекта OLE (Object Linking and Embedding, технология связывания и внедрения объектов).
- Гиперссылка. Обеспечивает связь с Web-страницей, расположенной в Internet, внутренней сети (intranet) или на локальном компьютере.
- Мастер подстановок. Это свойство поля, а не настоящий тип данных. Используется в программе Access для более эффективного и корректного ввода данных.

36. Организация запросов в MS Access.

SQL-запросы

С помощью языка структурированных запросов SQL, реализованного в Access 2000, можно составить любое число сложных запросов. Этот язык позволяет также управлять обработкой запросов.

SQL-запрос представляет собой последовательность инструкций, в которую могут входить выражения и статистические функции SQL.

Начинающим пользователям рекомендуется создавать запросы в окне конструктора, поскольку для описания запросов на языке SQL необходим опыт работы с базами данных. Примерами SQL-запросов являются запросы на объединение, запросы к серверу, управляющие и подчиненные запросы.

Некоторые запросы не могут быть сформированы в окне QBE-запроса. Разработка запросов на объединение, запросов к серверу и управляющих запросов осуществляется только в окне запроса в режиме SQL.

SQL-запросы могут служить не только для извлечения информации из базы данных, но и для получения сведений для формы и отчета, а также для заполнения полей списков в формах.

Рассмотрим важнейшие SQL-понятия.

SQL - это самый распространенный язык управления базами данных в системах клиент/сервер.

SQL-запрос составляется из последовательности *SQL-инструкций*, указывающих, что нужно сделать с *входным набором данных* (таблицей или запросом) для генерации *выходного набора*. При помощи аргументов (параметров) этих инструкций задают имена полей, имена таблиц, условия, отношения и т.п. Извлеченная из базы информация обрабатывается с помощью специальных *статистических функций plain*. При такой обработке можно определить, например, минимальное и максимальное значения, сумму и среднее значение.

SQL-инструкции

Инструкция SELECT

В основе большинства SQL-запросов лежит инструкция SELECT, важнейшие параметры которой приведены в следующем списке:

```
SELECT    Список_полей
FROM      Имена_таблиц
WHERE     Критерии_поиска
IN        Имя_базы_данных
ALL, DISTINCT, DISTINCTROW - предикаты
Таблица1 INNER JOIN Таблица2
GROUP BY  Список_полей

HAVING    Критерии_поиска
ORDER BY  Список_полей
```

37. Организация Макросов в MS Access.

Макросы

Access предоставляет пользователям два средства автоматизации работы с базами данных: язык макросов и язык Visual Basic for Applications (VBA). Эти средства позволяют превратить изнурительное повторение последовательностей операций в простую процедуру, состоящую в нажатии специальной кнопки или комбинации клавиш либо в активизации команды меню.

Язык VBA имеет преимущества перед языком макросов. Несмотря на это последний можно использовать для создания сложных программ и для автоматизации большинства операций по управлению базой данных.

38. Применение VBA в MS Access, основные термины языка.

Visual Basic for Applications

Access позволяет автоматизировать обработку баз данных двумя способами: посредством макросов и путем программирования на языке Visual Basic for Applications.

Поскольку возможности языка макросов ограничены, для создания сложных приложений следует использовать язык -Visual Basic for Applications (VBA), который доступен во всех приложениях Microsoft Office. Технология визуального программирования, реализованная в языке VBA, позволяет пользователям создавать приложения и самостоятельно решать многие задачи, не обращаясь к услугам профессиональных программистов.

39. Реализация автоматизированных информационных систем.

Реализация автоматизированных информационных систем

Важнейшая задача компьютерных систем - хранение и обработка данных. Для ее решения были предприняты усилия, которые привели к появлению в конце 60-х - начале 70-х годов специализированного программного обеспечения - систем управления базами данных (database management systems). СУБД позволяют структурировать, систематизировать и организовать данные для их компьютерного хранения и обработки.

1 История развития СУБД

Обращаясь к истории развития и совершенствования СУБД, можно условно выделить три основных этапа:

- 1) СУБД первого поколения опирались на иерархическую и сетевую модели данных. По времени они совпали с периодом, когда на рынке вычислительной техники доминировали большие ЭВМ (mainframe), например, система IBM 360, которые в совокупности с СУБД первого поколения составили аппаратно-программную платформу больших информационных систем.

- 2) С созданием реляционной модели данных был открыт новый этап в эволюции СУБД. Системы на основе РМД называют "вторым поколением СУБД". Его характеризовали две основные особенности - реляционное представление данных и язык запросов SQL (Structured Query Language).
- 3) СУБД третьего поколения - это сложные многофункциональные программные системы, работающие в открытой распределенной среде.

2 Технология реализации АИС состоит из следующих этапов:

1. Организация вычислительной среды
2. Организация вычислительного процесса
3. опытная эксплуатация
4. доработка автоматизированного продукта
5. ввод в эксплуатацию

Проблематика создания АИС:

- 1) Проблема обмена данных
 - 2) Проблема Связанная с распределенностью данных
 - 3) Проблема Репликации данных
 - 4) Проблема Интерфейса с пользователем
- 1) Проблема обмена (Наращивания) данных – наращивание информационной емкости БД связана с работой разных пользователей на 1 компьютере.

40. Отличительная особенность традиционного подхода к реализации АИС, проблема наращивания данных.

1-ая проблема: наращивание информационной емкости БД – связана с работой разных пользователей на 1 компьютере.

Дейт обобщил требования в виде 12 правил:

1. пространственная прозрачность – директивы не должны учитывать место хранения данных
2. прозрачность распределения – способ распределения не должен влиять на команды (это относится к средствам автоматического согласования изменений)
3. Полная функциональность – все операции в распределенной БД должны быть возможны также как и в локальной БД
4. Целостность данных во всей сети
5. Независимость от оборудования
6. Независимость от СУБД

41. Отличительная особенность традиционного подхода к реализации.

АИС, проблема распределённости данных.

2-ая проблема: **распределённость данных** - анализ вариантов технологической реализации АИС с учетом видов компонент.

42. СУБД в архитектуре "клиент-сервер".

Технология и модели "клиент-сервер".

37

"Клиент-сервер" - это модель взаимодействия компьютеров в сети.

Применительно к СУБД, архитектура "клиент-сервер" актуальна потому, что обеспечивает простое и относительно дешевое решение проблемы коллективного доступа к базам данных в локальной сети.

Для большинства функций СУБД существуют некоторые аналоги на уровне файловых систем. А вот наличие нескольких способов работы с данными является принципиальным отличием СУБД от файлов. Выбор той или иной СУБД на практике часто обуславливается именно удобством работы с ней, а не только производительностью, надёжностью и совершенством архитектуры. Прямой доступ Аналогом доступа к файлам можно считать т.н. прямой доступ к СУБД – либо через дополнительные библиотеки того языка, на котором написана СУБД, либо через специальный язык программирования. Большим преимуществом прямого доступа, безусловно, является высокая производительность. Но так как собственные языки или библиотеки довольно сложны и пригодны для работы лишь с одной СУБД, обычно прямой доступ используют лишь сами разработчики этой СУБД для написания законченных приложений – “дизайнеров”, “навигаторов”, “менеджеров” и других графических оболочек БД. К сожалению, такие приложения можно рассматривать лишь как средства отладки, они неудобны для конечных пользователей, поскольку они не отражают специфику их данных. Языки баз данных В связи с этим понятно, почему любая СУБД должна поддерживать хотя бы один универсальный интерфейс, через который можно

обращаться к ней из приложений, написанных другими программистами. Ближе всего к прямому доступу находятся интерфейсы, называемые языками баз данных. Их операторы компилируются (в процессе работы с БД или заранее) в исполняемый код специфического для каждой СУБД языка. Языки баз данных раньше делились на языки определения данных (языки задания структуры) и языки манипулирования данными, однако сейчас обе категории операций с БД объединены в интегрированных языках типа SQL (Structured Query Language). Эти языки стандартизованы, хотя есть различия в реализации языков разными СУБД, и поэтому пока очень редко бывает, чтобы приложение, работавшее с одной СУБД, без изменений могло работать с другой. Инструкции языков баз данных можно последовательно исполнять в специальных программах типа Oracle SQL Plus или Microsoft Query. Интерфейсы уровня обращения и их производные. Однако чаще всего языки баз данных используются в так называемых интерфейсах уровня обращения. Для современных языков программирования существуют библиотеки (драйверы), которые позволяют обращаться через эти интерфейсы к СУБД, пересылая туда инструкции языков баз данных и получая результаты некоторых запросов в виде специальных структур языка программирования. Производители СУБД поставляют свои библиотеки CLI, оптимизированные для их продукта, однако в последнее время наметилась тенденция к стандартизации этих библиотек. В мире реляционных СУБД наиболее распространены следующие стандартные протоколы программного доступа к БД: ODBC от Microsoft и JDBC от Sun, а также OCI от Oracle. Если программисты используют стандартный интерфейс уровня обращения и придерживаются стандарта языка баз данных, то их приложение сможет работать с разными СУБД. В некоторых средствах программирования существуют также библиотеки, основанные на интерфейсах уровня обращения, которые существенно проще в использовании по сравнению с этими интерфейсами. Примерами могут служить компоненты DAO от Microsoft, Data Express от Borland, или OCI/OCA от Oracle. При использовании таких библиотек почти не требуется знания языков баз данных. Однако в качестве платы за такое упрощение программист вынужден ограничиться небольшим набором СУБД, которые поддерживаются поставщиками библиотек. Ещё сильнее зависит от конкретной

СУБД другой способ программного доступа к БД – встраиваемые в код языка программирования инструкции языка баз данных. Собственные языки программирования, имеющиеся для некоторых СУБД, всегда снабжены синтаксисом встраивания, а производители других СУБД иногда поставляют трансляторы для исходного кода существующих языков, которые заменяют встроенные инструкции языка баз данных на вызовы процедур из специальных библиотек. В обоих случаях набор операций, которые необходимо производить с данными, определяется на этапе компиляции программы, и поэтому рассматриваемый способ доступа к данным называется статическим. Трансляторы встроенных инструкций проверяют их синтаксис и наличие указанных в них логических объектов в конкретной базе данных, а также соответствие между типами данных языка программирования и языка базы данных. Объектные интерфейсы

Выше речь шла, прежде всего, о реляционных интерфейсах доступа к БД. Во многих современных СУБД поддерживаются также объектные интерфейсы доступа, которые пока имеют менее сложившийся стандарт, чем интерфейсы уровня обращения. Поэтому многие объектно-реляционные СУБД имеют собственные библиотеки, которые обеспечивают автоматическую загрузку и сохранение программных объектов в БД, и лишь немногие СУБД полностью поддерживают стандарт ODMG. Объектный доступ отличается от реляционного тем, что структуры данных, которыми обменивается программа с СУБД, менее формализованы и фактически совпадают со структурами данных этой программы; поэтому не приходится проводить сложные преобразования результатов каждого запроса к СУБД. Другими словами, работать с хранимыми в базе данных объектами почти так же легко, как с нехранимыми структурами данных языка программирования; необходимо лишь наследовать классы этих объектов от классов написанной другими программистами библиотеки и вызывать некоторые их методы. Встраивание объектных библиотек в приложения может осуществляться не только вручную, но и с помощью утилит, которые устанавливают соответствие между структурами базы данных и структурами объектно-ориентированного языка программирования – классами. Это соответствие может быть как прямым, так и обратным. Обычно утилиты такого типа поставляются разработчиками СУБД; примером “двунаправленной” и

независящей от СУБД утилиты является продукт Sun JavaBlend, который преобразует объекты Java в объекты БД и обратно через реляционный интерфейс JDBC. В случае использования объектов абстрагироваться от конкретной СУБД позволяет программное обеспечение промежуточного уровня. Оно избавляет программистов от необходимости знания языков баз данных, интерфейсов уровня обращения или специфических для конкретных СУБД объектных библиотек. При этом разные объекты могут даже храниться в различных СУБД, что обеспечивает программа промежуточного уровня, а для разработчиков конечного приложения важна лишь возможность загрузки и сохранения этих объектов. Достоинством трёхуровневой архитектуры по сравнению с архитектурой “клиент-сервер” является возможность более быстрого создания и развития приложений, а к недостаткам – низкая производительность. К промежуточному уровню относятся программы, реализующие распределённые объектные интерфейсы – DCOM, Enterprise JavaBeans. Соотношение реляционных и объектных интерфейсов Как можно заметить из вышеизложенного, доступ к реляционным БД лучше всего поддаётся стандартизации на низком уровне – на уровне языков баз данных, а на высоком уровне middleware крайне затруднителен. И наоборот, доступ к объектным БД на уровне языка базы данных плохо стандартизован (пока на роль стандарта претендуют два языка – SQL и OQL. Зато использование высокоуровневых распределённых объектных интерфейсов делает приложение совершенно независимым от СУБД. Вообще, роль реляционных и объектных интерфейсов в современных СУБД весьма разнообразна. Исторически сложилось так, что самые уважаемые СУБД имеют реляционное ядро, которое впоследствии дополнилось объектными надстройками. С другой стороны, относительно “молодые” СУБД имеют объектное ядро, по возможности оснащённое реляционным интерфейсом, а постреляционные СУБД вообще, основана на т.н. многомерной модели данных, по отношению к которой и объектный, и реляционный интерфейсы являются внешними. Естественно, надстройки часто понижают производительность СУБД, что следует учитывать при выборе оптимальной комбинации “СУБД – интерфейс доступа – язык программирования”. Другими словами, с объектной СУБД быстрее работать через объектный интерфейс, с реляционной – через реляционный интерфейс, хотя в

случае многомерной СУБД, ориентированной на OLAP-приложения, производительность не “родных” для неё объектных или реляционных запросов на выборку данных тоже достаточно высока (конечно, за счёт понижения скорости запросов на изменение данных). Web-интерфейс Какими бы удобными для программистов ни были объектные интерфейсы доступа, они требуют некоторых усилий по представлению хранимых в БД объектов пользователю. В частности, при написании Internet-приложений (Java-апплетов) программист отвечает за преобразование получаемых из СУБД результатов запросов и размещение их в графических компонентах апплета; при этом также велика нагрузка на сеть (поскольку запросы являются элементарными, их приходится делать много). Использование вместо архитектуры “клиент-сервер” трёхуровневой архитектуры этих проблем не решает. Задача сильно упрощается, если СУБД возвращает результаты запросов к ней вместе с атрибутами их визуального представления, а программисту остаётся лишь задать правила для формирования этих атрибутов. На практике эти результаты отображаются Web-браузером в виде страницы на одном из языков разметки (HTML или XML), что даёт дополнительные преимущества: СУБД выходит за рамки локальной сети в глобальную сеть, а пользователи, даже если они находятся в локальной сети с сервером СУБД, работают в привычных для них программах MS Internet Explorer или Netscape Navigator. Как правило, за преобразование данных из формата СУБД (обычно это формат интерфейсов уровня обращения) отвечают специальные программы, написанные на скриптовых языках (PHP, ASP, Perl) и реализующие интерфейс CGI (Common Gateway Interface). Эти программы взаимодействуют с Web-сервером, получая через него запросы Web-браузера (в виде строк URL) и возвращая обратно данные из СУБД (в виде HTML-страниц). Сетевой трафик при этом меньше, чем при прямой связи клиентского апплета с СУБД. Однако использование внешнего по отношению к СУБД CGI-скрипта – дополнительного звена в и без того длинной цепочке передачи данных – плохо сказывается на производительности, поэтому желательно, чтобы СУБД имела свой Web-интерфейс. При этом, правда, теряется стандартность Web-доступа: каждая такая СУБД будет иметь свой язык скриптов, не соответствующий CGI. Доступ к базе данных от прикладной программы или пользователя производится путем

обращения к клиентской части системы. В качестве основного интерфейса между клиентской и серверной частями выступает язык баз данных SQL. Это язык по сути дела представляет собой текущий стандарт интерфейса СУБД в открытых системах. Собирательное название SQL-сервер относится ко всем серверам баз данных, основанных на SQL. Соблюдая предосторожности при программировании, некоторые из которых были рассмотрены на предыдущих лекциях, можно создавать прикладные информационные системы, мобильные в классе SQL-серверов. Серверы баз данных, интерфейс которых основан исключительно на языке SQL, обладают своими преимуществами и своими недостатками. Очевидное преимущество - стандартность интерфейса. В пределе, хотя пока это не совсем так, клиентские части любой SQL-ориентированной СУБД могли бы работать с любым SQL-сервером вне зависимости от того, кто его произвел. Недостаток тоже довольно очевиден. При таком высоком уровне интерфейса между клиентской и серверной частями системы на стороне клиента работает слишком мало программ СУБД. Это нормально, если на стороне клиента используется маломощная рабочая станция. Но если клиентский компьютер обладает достаточной мощностью, то часто возникает желание возложить на него больше функций управления базами данных, разгрузив сервер, который является узким местом всей системы. Одним из перспективных направлений СУБД является гибкое конфигурирование системы, при котором распределение функций между клиентской и пользовательской частями СУБД определяется при установке системы. Преимущества протоколов удаленного вызова процедур Упомянувшиеся выше протоколы удаленного вызова процедур особенно важны в системах управления базами данных, основанных на архитектуре "клиент-сервер". Во-первых, использование механизма удаленных процедур позволяет действительно перераспределять функции между клиентской и серверной частями системы, поскольку в тексте программы удаленный вызов процедуры ничем не отличается от удаленного вызова, теоретически любой компонент системы может располагаться и на стороне сервера, и на стороне клиента. Во-вторых, механизм удаленного вызова скрывает различия между взаимодействующими компьютерами. Физически неоднородная локальная сеть компьютеров приводится к логически однородной сети взаимодействующих программных

компонентов. В результате пользователи не обязаны серьезно заботиться о разовой закупке совместимых серверов и рабочих станций. Типичное разделение функций между клиентами и серверами В типичном на сегодняшний день случае на стороне клиента СУБД работает только такое программное обеспечение, которое не имеет непосредственного доступа к базам данных, а обращается для этого к серверу с использованием языка SQL. В некоторых случаях хотелось бы включить в состав клиентской части системы некоторые функции для работы с "локальным кэшем" базы данных, т.е. с той ее частью, которая интенсивно используется клиентской прикладной программой. В современной технологии это можно сделать только путем формального создания на стороне клиента локальной копии сервера базы данных и рассмотрения всей системы как набора взаимодействующих серверов. С другой стороны, иногда хотелось бы перенести большую часть прикладной системы на сторону сервера, если разница в мощности клиентских рабочих станций и сервера чересчур велика. В общем-то при использовании RPC это сделать нетрудно. Но требуется, чтобы базовое программное обеспечение сервера действительно позволяло это. В частности, при использовании ОС UNIX проблемы практически не возникают.

43. Серверы баз данных, принципы взаимодействия между клиентскими и серверными частями.

Сервер БД обслуживает базу данных и отвечает за целостность и сохранность данных, а также обеспечивает операции ввода-вывода при доступе клиента к информации.

Архитектура клиент-сервер состоит из клиентов и серверов. Основная идея состоит в том, чтобы размещать серверы на мощных машинах, а приложениям, использующим языковые компоненты СУБД, обеспечить доступ к ним с менее мощных машин-клиентов посредством внешних интерфейсов.

Содержание

- Язык SQL
- Хранимые и присоединенные процедуры
- Операционная среда серверов
- Ссылочная целостность реляционных БД
- Транзакции и целостность БД
- Согласованность чтения
- Тупиковые ситуации

- Схемы оптимизации работ на языке SQL

Язык SQL

Большинство СУБД используют язык SQL (Structured Query Language — язык структурированных запросов), так как он удобен для описания логических подмножеств БД.

Назначение SQL:

- создание БД и таблицы с полным описанием их структуры;
- выполнение основных операций манипулирования данными (такие как вставка, модификация и удаление данных из таблиц);
- выполнение простых и сложных запросов.

Одна из ключевых особенностей языка SQL заключается в том, что с его помощью формируются запросы, описывающие какую информацию из базы данных необходимо получить, а пути решения этой задачи программа определяет сама.

Хранимые и присоединенные процедуры

Существуют расширенные версии языка SQL, которые поддерживают такие расширения, как хранимые и расширенные процедуры, а также управление ходом программы через ветвления и организацию циклов. Хранимые процедуры — это предварительно откомпилированные предложения языка SQL, которые сохраняются на сервере базы данных, использующей язык SQL. Клиент запускает хранимую процедуру с помощью команды EXECUTE<имя процедуры>. Таким образом, по сети передаются только два слова вместо двух сотен. Поскольку эта процедура уже откомпилирована и оптимизирована, серверу не нужно тратить время на компиляцию и оптимизацию.

В качестве хранимых процедур обычно используются часто выполняемые запросы. Присоединенные процедуры (триггеры) подобны хранимым процедурам и исполняются в ответ на события, происходящие в БД. Когда с некоторыми приложениями языка SQL связана присоединенная процедура, выполнение этого предложения всегда запускает целую серию команд, входящих в эту процедуру. Присоединенная процедура автоматически выполняет одно или более предложений языка SQL, всякий раз, когда выполняет предложения INSERT, UPDATE или DELETE. Самое важное применение присоединенных процедур заключается в обеспечении ссылочной целостности.

Операционная среда серверов

Серверы БД: SQL SERVER (Microsoft), SQL BASE SERVER, Oracle SERVER (Oracle Corporation). Каждый сервер БД может работать на определенных типах компьютеров и сетей. Операционными системами серверов могут быть MSDOS, OS/2, Xenix, Unix, Dec VMS/ Рабочие станции пользователей обычно работают под управление MSDOS, OS/2, Xenix, Unix. Существуют возможности смешанного

использования различных ОС. Большая часть SQL-серверов может хранить описание БД в системном каталоге, который обычно бывает доступен пользователям. Для обращения к этому каталогу используются SQL-запросы. Реляционные СУБД могут использовать информацию, хранящуюся в системном каталоге для оптимизации SQL-запросов.

Ссылочная целостность реляционных БД

Большинство SQL-серверов поддерживают ссылочную целостность реляционных БД, состоящих из отдельных таблиц, которые могут быть объединены на основе общей информации. Рассмотрим на следующем примере: база данных содержит таблицу клиентов и таблицу заказов, которые связаны полем номера клиента, содержащимся в обеих таблицах. Поскольку может быть более одного заказа от одного клиента, соотношение таблиц — «один-ко-многим». Когда таблицы соединены, то таблица клиентов является родительской, а таблица заказов — дочерней. Если запись-родитель стирается, а соответствующие ей дочерние записи — нет, то говорят, что дочерние записи «осиротели». Ссылочная целостность означает, что ни в одной таблице не допустимы записи-«сироты». Запись может осиротеть тремя способами:

- 1)родительская запись удалена;
- 2)родительская запись изменена таким образом, что связь между «родителем» и «потомками» потеряна;
- 3)введена дочерняя запись без соответствующей родительской.

Поддержание ссылочной целостности возможно несколькими способами:

- 1)Через ключи, хранящиеся в таблицах БД (родительские таблицы содержат первичные ключи, представляющие собой комбинации внешних ключей, которые могут быть найдены внутри каждой из дочерних таблиц).
- 2)Использование присоединенных процедур — процедурная ссылочная целостность. Присоединенные программы обеспечивают ссылочную целостность за счет автоматического выполнения предложений SQL всякий раз, когда встречается одно из предложений UPDATE/INSERT или DELETE (либо запрещается удаление родительской записи, либо стираются все дочерние записи).

44. Проблема тиражирования данных.

Способ распространения изменённых данных.

Благодаря тиражированию(копированию репликаций) пользователи могут более оперативно обращаться к локальным данным. При этом необходимо обеспечить целостность данных, и механизм их управления. Существуют 2 технологии тиражирования

1. Синхронное (двухфазная фиксация) - предполагают практически одновременное изменение всех данных во всех узлах системы. Если один

из узлов недоступен, то тиражирование отменяется. Синхронное тиражирование является более сложным в реализации, но при нём невозможен случай когда в системе существуют различные копии одних и тех же данных. Перед обновлением основной копии данных на сервере, он отправляет всем клиентам команду блокировки данных. После чего сервер рассылает изменённую копию данных для внесения изменений в локальные копии. После внесения изменений, каждый клиент отправляет серверу подтверждение, если сервером были получены подтверждения от всех клиентов, то сервер отправляет команду фиксации транзакции. Если хотя бы от одного из клиентов подтверждение не пришло, то тиражирование данных отменяется. После завершения тиражирования, блокировка с данных снимается. Синхронное тиражирование применяется для обеспечения согласованности и целостности данных, и различные его реализации присутствуют практически во всех корпоративных СУБД. А при большом количестве узлов (>50) добиться того чтобы все клиенты изменили свою копию данных практически невозможно, но если это получается, то время на выполнение тиражирования достаточно велико. Условиями применимости синхронного тиражирования являются:

- a. Изменения должны и могут распространяться немедленно.
 - b. Необходимо обеспечить высокую степень целостности данных.
 - c. Необходимо обеспечить абсолютную согласованность данных. (Ни одна копия не должна иметь старую копию)
2. Асинхронное – В силу недостатков синхронного тиражирования, в последнее время всё большую популярность приобретает асинхронное тиражирование, которое предполагает копирование данных на локальные машины, с последующим их изменением. При асинхронном тиражировании, сначала изменяется основная копия на сервере, а уже потом изменение вносится в локальные копии. Если одна из таких копий оказывается не доступной, то изменения в неё будут внесены позже. В первую очередь обновляются доступные копии. Если к окончанию процесса тиражирования остались не обновлённые копии, то для их изменения требуется вмешательство администратора. Может применяться в 2 случаях:
- a. Изменения не должны распространяться немедленно.
 - b. Характер приложения должен быть таким, чтобы не требовалось сообщать о малейшей согласованности данных.

Тиражирование осуществляется с помощью 3 архитектурных схем:

1. Главный подчинённый (которая допускает реализацию 2 вариантов)
 - a. Тиражирование из основного узла. (данные копируются из основной таблицы в удалённые таблицы, доступные только для чтения при этом изменение данных происходит только в основной таблице. То есть основная таблица является владельцем данных, а удалённые таблицы её копиями. Когда клиенту надо что-то изменить, он обращается к владельцу данных)
 - b. Горячее резервирование (Данный вариант предполагает копирование данных не только на клиентах, но и на второй вспомогательный сервер, который в случае выхода из строя первого сервера, играет роль основного)

2. Равноправные узлы либо одноранговые схемы.(Данными владеют все узлы, и поэтому вносить изменения могут так же все узлы, при этом необходимо разрешать конфликты связанные с поступлением нескольких запросов на изменение. Например посредством приоритетов.)
3. Гибридная схема(Владелец данных меняется динамически, данные могут изменяться на любых узлах сети, но в каждый фиксированный момент времени это может делать только один узел.)

В распределённых вычислительных системах часто возникает проблема синхронизации данных, хранящихся на различных компьютерах.

Механизмы тиражирования данных, должны обеспечить либо целостность данных, либо их автономную работу.

1. Тиражирование слиянием.(максимальная автономность отдельных копий данных и суть его заключается в том что при изменении копии локальной СУБД сохраняют все команды над данными и их последовательность. В некоторый момент времени, происходит подключение к главному узлу, и передача произведённых над данными команд в этот узел.) Пример: Организация в которой, сотрудники выезжают на встречи с клиентом для заключения контрактов, при этом работа производится в автономном режиме. Как только сотрудник приезжает в главный офис, он подключается к серверу, передаёт ему все изменения данных и получает обновленные данные от сервера.
2. Тиражирование моментальными снимками.(тиражируются не команды над данными, а снимок всех данных в том числе и системных. Такой механизм может быть использован в системах, в которых клиентам данные доступны только для чтения, все изменения данных вносятся через данный сервер. Такая схема изменений данных достаточно проста не требует разрешения конфликтов и может быть использована в случае, когда данные меняются достаточно редко)Пример: Распространение цен на товары в магазинах.
3. Тиражирование транзакций.(Сервером к клиентам распространяются не отдельные команды, а целые транзакции, при этом с некоторым интервалом, необходимо производить синхронизацию данных, с помощью полного снимка БД. Лучше всего использовать, когда в системе много данных, а операций над ними выполняется мало.)
4. Механизм немедленного обновления. (Как только на каком либо клиенте произошло изменение данных, эти изменения будут моментально переданы остальным клиентам. А примером использования такого механизма тиражирования является продажа билетов, когда после продажи одного билета, необходимо сообщить информацию об этом остальным клиентам)
5. Распределённые транзакции.(В случае изменения данных, сервер инициирует распределённую транзакцию, в которую привлекаются все клиенты, а после изменения локальных копий, клиенты сообщают серверу об этом, и сервер посылает команду – зафиксировать транзакцию. Если один из клиентов, не подтвердил, то транзакция отменяется.

45. Хеширование (хеш-адресация).

Если коротко, то криптографическая хеш-функция, чаще называемая просто хешем, — это математический алгоритм, преобразовывающий произвольный

массив данных в состоящую из букв и цифр строку фиксированной длины. Причем при условии использования того же типа хеша длина эта будет оставаться неизменной, вне зависимости от объема входных данных. Криптостойкой хеш-функция может быть только в том случае, если выполняются главные требования: стойкость к восстановлению хешируемых данных и стойкость к коллизиям, то есть образованию из двух разных массивов данных двух одинаковых значений хеша. Интересно, что под данные требования формально не подпадает ни один из существующих алгоритмов, поскольку нахождение обратного хешу значения — вопрос лишь вычислительных мощностей. По факту же в случае с некоторыми особо продвинутыми алгоритмами этот процесс может занимать чудовищно много времени. Если коротко, то криптографическая хеш-функция, чаще называемая просто хешем, — это математический алгоритм, преобразовывающий произвольный массив данных в состоящую из букв и цифр строку фиксированной длины. Причем при условии использования того же типа хеша длина эта будет оставаться неизменной, вне зависимости от объема входных данных. Криптостойкой хеш-функция может быть только в том случае, если выполняются главные требования: стойкость к восстановлению хешируемых данных и стойкость к коллизиям, то есть образованию из двух разных массивов данных двух одинаковых значений хеша. Интересно, что под данные требования формально не подпадает ни один из существующих алгоритмов, поскольку нахождение обратного хешу значения — вопрос лишь вычислительных мощностей. По факту же в случае с некоторыми особо продвинутыми алгоритмами этот процесс может занимать чудовищно много времени. Например, мое имя — Brian — после преобразования хеш-функцией SHA-1 (одной из самых распространенных наряду с MD5 и SHA-2) при помощи онлайн-генератора будет выглядеть так: 75c450c3f963befb912ee79f0b63e563652780f0. Как вам скажет, наверное, любой другой Брайан, данное имя нередко пишут с ошибкой, что в итоге превращает его в слово brain (мозг). Это настолько частая опечатка, что однажды я даже получил настоящие водительские права, на которых вместо моего имени красовалось Brain Donohue. Впрочем, это уже другая история. Так вот, если снова воспользоваться алгоритмом SHA-1, то слово Brain трансформируется в строку 97fb724268c2de1e6432d3816239463a6aaf8450. Как видите, результаты значительно

отличаются друг от друга, даже несмотря на то, что разница между моим именем и названием органа центральной нервной системы заключается лишь в последовательности написания двух гласных. Более того, если я преобразую тем же алгоритмом собственное имя, но написанное уже со строчной буквы, то результат все равно не будет иметь ничего общего с двумя предыдущими: 760e7dab2836853c63805033e514668301fa9c47. Впрочем, кое-что общее у них все же есть: каждая строка имеет длину ровно 40 символов. Казалось бы, ничего удивительного, ведь все введенные мною слова также имели одинаковую длину — 5 букв. Однако если вы захешируете весь предыдущий абзац целиком, то все равно получите последовательность, состоящую ровно из 40 символов: c5e7346089419bb4ab47aaa61ef3755d122826e2. То есть 1128 символов, включая пробелы, были ужаты до строки той же длины, что и пятибуквенное слово. То же самое произойдет даже с полным собранием сочинений Уильяма Шекспира: на выходе вы получите строку из 40 букв и цифр. При всем этом не может существовать двух разных массивов данных, которые преобразовывались бы в одинаковый хеш. Если коротко, то криптографическая хеш-функция, чаще называемая просто хешем, — это математический алгоритм, преобразовывающий произвольный массив данных в состоящую из букв и цифр строку фиксированной длины. Причем при условии использования того же типа хеша длина эта будет оставаться неизменной, вне зависимости от объема входных данных. Криптостойкой хеш-функция может быть только в том случае, если выполняются главные требования: стойкость к восстановлению хешируемых данных и стойкость к коллизиям, то есть образованию из двух разных массивов данных двух одинаковых значений хеша. Интересно, что под данные требования формально не подпадает ни один из существующих алгоритмов, поскольку нахождение обратного хешу значения — вопрос лишь вычислительных мощностей. По факту же в случае с некоторыми особо продвинутыми алгоритмами этот процесс может занимать чудовищно много времени. Например, мое имя — Brian — после преобразования хеш-функцией SHA-1 (одной из самых распространенных наряду с MD5 и SHA-2) при помощи онлайн-генератора будет выглядеть так: 75c450c3f963befb912ee79f0b63e563652780f0. Как вам скажет, наверное, любой другой Брайан, данное имя нередко пишут с ошибкой, что в итоге превращает его в слово brain (мозг). Это настолько частая опечатка, что

однажды я даже получил настоящие водительские права, на которых вместо моего имени красовалось Brain Donohue. Впрочем, это уже другая история. Так вот, если снова воспользоваться алгоритмом SHA-1, то слово Brain трансформируется в строку 97fb724268c2de1e6432d3816239463a6aaf8450. Как видите, результаты значительно отличаются друг от друга, даже несмотря на то, что разница между моим именем и названием органа центральной нервной системы заключается лишь в последовательности написания двух гласных. Более того, если я преобразую тем же алгоритмом собственное имя, но написанное уже со строчной буквы, то результат все равно не будет иметь ничего общего с двумя предыдущими: 760e7dab2836853c63805033e514668301fa9c47. Впрочем, кое-что общее у них все же есть: каждая строка имеет длину ровно 40 символов. Казалось бы, ничего удивительного, ведь все введенные мною слова также имели одинаковую длину — 5 букв. Однако если вы захешируете весь предыдущий абзац целиком, то все равно получите последовательность, состоящую ровно из 40 символов: c5e7346089419bb4ab47aaa61ef3755d122826e2. То есть 1128 символов, включая пробелы, были ужалы до строки той же длины, что и пятибуквенное слово. То же самое произойдет даже с полным собранием сочинений Уильяма Шекспира: на выходе вы получите строку из 40 букв и цифр. При всем этом не может существовать двух разных массивов данных, которые преобразовывались бы в одинаковый хеш. Отличный вопрос. Однако ответ не так прост, поскольку криптохешы используются для огромного количества вещей. Для нас с вами, простых пользователей, наиболее распространенная область применения хеширования — хранение паролей. К примеру, если вы забыли пароль к какому-либо онлайн-сервису, скорее всего, придется воспользоваться функцией восстановления пароля. В этом случае вы, впрочем, не получите свой старый пароль, поскольку онлайн-сервис на самом деле не хранит пользовательские пароли в виде обычного текста. Вместо этого он хранит их в виде хеш-значений. То есть даже сам сервис не может знать, как в действительности выглядит ваш пароль. Исключение составляют только те случаи, когда пароль очень прост и его хеш-значение широко известно в кругах взломщиков. Таким образом, если вы, воспользовавшись функцией восстановления, вдруг получили старый пароль в открытом виде, то можете быть уверены: используемый вами сервис не хеширует пользовательские пароли, что очень плохо.

Вы даже можете провести простой эксперимент: попробуйте при помощи специального сайта произвести преобразование какого-нибудь простого пароля вроде «123456» или «password» из их хеш-значений (созданных алгоритмом MD5) обратно в текст. Вероятность того, что в базе хешей найдутся данные о введенных вами простых паролях, очень высока. В моем случае хеши слов «brain» (8b373710bcf876edd91f281e50ed58ab) и «Brian» (4d236810821e8e83a025f2a83ea31820) успешно распознались, а вот хеш предыдущего абзаца — нет. Отличный пример, как раз для тех, кто все еще использует простые пароли. Еще один пример, покруче. Не так давно по тематическим сайтам прокатилась новость о том, что популярный облачный сервис Dropbox заблокировал одного из своих пользователей за распространение контента, защищенного авторскими правами. Герой истории тут же написал об этом в твиттере, запустив волну негодования среди пользователей сервиса, ринувшихся обвинять Dropbox в том, что он якобы позволяет себе просматривать содержимое клиентских аккаунтов, хотя не имеет права этого делать. Впрочем, необходимости в этом все равно не было. Дело в том, что владелец защищенного копирайтом контента имел на руках хеш-коды определенных аудио- и видеофайлов, запрещенных к распространению, и занес их в список блокируемых хешей. Когда пользователь предпринял попытку незаконно распространить некий контент, автоматические сканеры Dropbox засекли файлы, чьи хеши оказались в пресловутом списке, и заблокировали возможность их распространения. Где еще можно использовать хеш-функции помимо систем хранения паролей и защиты медиафайлов? На самом деле задач, где используется хеширование, гораздо больше, чем я знаю и тем более могу описать в одной статье. Однако есть одна особенная область применения хешей, особо близкая нам как сотрудникам «Лаборатории Касперского»: хеширование широко используется для детектирования зловредных программ защитным ПО, в том числе и тем, что выпускается нашей компанией.

Если коротко, то криптографическая хеш-функция, чаще называемая просто хешем, — это математический алгоритм, преобразовывающий произвольный массив данных в состоящую из букв и цифр строку фиксированной длины. Причем при условии использования того же типа хеша длина эта будет оставаться неизменной, вне зависимости от объема входных данных. Криптостойкой хеш-

функция может быть только в том случае, если выполняются главные требования: стойкость к восстановлению хешируемых данных и стойкость к коллизиям, то есть образованию из двух разных массивов данных двух одинаковых значений хеша. Интересно, что под данные требования формально не подпадает ни один из существующих алгоритмов, поскольку нахождение обратного хешу значения — вопрос лишь вычислительных мощностей. По факту же в случае с некоторыми особо продвинутыми алгоритмами этот процесс может занимать чудовищно много времени. Например, мое имя — Brian — после преобразования хеш-функцией SHA-1 (одной из самых распространенных наряду с MD5 и SHA-2) при помощи онлайн-генератора будет выглядеть так: 75c450c3f963befb912ee79f0b63e563652780f0. Как вам скажет, наверное, любой другой Брайан, данное имя нередко пишут с ошибкой, что в итоге превращает его в слово brain (мозг). Это настолько частая опечатка, что однажды я даже получил настоящие водительские права, на которых вместо моего имени красовалось Brain Donohue. Впрочем, это уже другая история. Так вот, если снова воспользоваться алгоритмом SHA-1, то слово Brain трансформируется в строку 97fb724268c2de1e6432d3816239463a6aaf8450. Как видите, результаты значительно отличаются друг от друга, даже несмотря на то, что разница между моим именем и названием органа центральной нервной системы заключается лишь в последовательности написания двух гласных. Более того, если я преобразую тем же алгоритмом собственное имя, но написанное уже со строчной буквы, то результат все равно не будет иметь ничего общего с двумя предыдущими: 760e7dab2836853c63805033e514668301fa9c47. Впрочем, кое-что общее у них все же есть: каждая строка имеет длину ровно 40 символов. Казалось бы, ничего удивительного, ведь все введенные мною слова также имели одинаковую длину — 5 букв. Однако если вы захешируете весь предыдущий абзац целиком, то все равно получите последовательность, состоящую ровно из 40 символов: c5e7346089419bb4ab47aaa61ef3755d122826e2. То есть 1128 символов, включая пробелы, были ужаты до строки той же длины, что и пятибуквенное слово. То же самое произойдет даже с полным собранием сочинений Уильяма Шекспира: на выходе вы получите строку из 40 букв и цифр. При всем этом не может существовать двух разных массивов данных, которые преобразовывались бы в одинаковый хеш. Отличный вопрос. Однако ответ не так прост, поскольку криптохеши

используются для огромного количества вещей. Для нас с вами, простых пользователей, наиболее распространенная область применения хеширования — хранение паролей. К примеру, если вы забыли пароль к какому-либо онлайн-сервису, скорее всего, придется воспользоваться функцией восстановления пароля. В этом случае вы, впрочем, не получите свой старый пароль, поскольку онлайн-сервис на самом деле не хранит пользовательские пароли в виде обычного текста. Вместо этого он хранит их в виде хеш-значений. То есть даже сам сервис не может знать, как в действительности выглядит ваш пароль. Исключение составляют только те случаи, когда пароль очень прост и его хеш-значение широко известно в кругах взломщиков. Таким образом, если вы, воспользовавшись функцией восстановления, вдруг получили старый пароль в открытом виде, то можете быть уверены: используемый вами сервис не хеширует пользовательские пароли, что очень плохо. Вы даже можете провести простой эксперимент: попробуйте при помощи специального сайта произвести преобразование какого-нибудь простого пароля вроде «123456» или «password» из их хеш-значений (созданных алгоритмом MD5) обратно в текст. Вероятность того, что в базе хешей найдутся данные о введенных вами простых паролях, очень высока. В моем случае хеши слов «brain» (8b373710bcf876edd91f281e50ed58ab) и «Brian» (4d236810821e8e83a025f2a83ea31820) успешно распознались, а вот хеш предыдущего абзаца — нет. Отличный пример, как раз для тех, кто все еще использует простые пароли. Еще один пример, покруче. Не так давно по тематическим сайтам прокатилась новость о том, что популярный облачный сервис Dropbox заблокировал одного из своих пользователей за распространение контента, защищенного авторскими правами. Герой истории тут же написал об этом в твиттере, запустив волну негодования среди пользователей сервиса, ринувшихся обвинять Dropbox в том, что он якобы позволяет себе просматривать содержимое клиентских аккаунтов, хотя не имеет права этого делать. Впрочем, необходимости в этом все равно не было. Дело в том, что владелец защищенного копирайтом контента имел на руках хеш-коды определенных аудио- и видеофайлов, запрещенных к распространению, и занес их в список блокируемых хешей. Когда пользователь предпринял попытку незаконно распространить некий контент, автоматические сканеры Dropbox засекли файлы, чьи хеши оказались в пресловутом

списке, и заблокировали возможность их распространения. Где еще можно использовать хеш-функции помимо систем хранения паролей и защиты медиафайлов? На самом деле задач, где используется хеширование, гораздо больше, чем я знаю и тем более могу описать в одной статье. Однако есть одна особенная область применения хешей, особо близкая нам как сотрудникам «Лаборатории Касперского»: хеширование широко используется для детектирования зловредных программ защитным ПО, в том числе и тем, что выпускается нашей компанией. Идеальный вариант – уметь по имени сразу определить адрес элемента. В этом смысле "хорошей" процедурой поиска является та, которая сможет определить хотя бы приблизительный адрес элемента. Одним из наиболее эффективных и распространенных методов реализации такой процедуры является хеш-адресация. Хеш-адресация – это метод преобразования имени в индекс элемента в таблице. Если таблица состоит из N элементов, то им присваиваются номера $0, 1, \dots, N-1$. Тогда назовем хеш-функцией некоторое преобразование имени в адрес. Простейший вариант хеш-функции – это использование внутреннего представления первой литеры имени. Пока для двух различных символов результаты хеширования различны, время поиска совпадает с временем, затраченным на хеширование. Однако, все проблемы начинаются тогда, когда результаты хеширования совпадают для двух и более различных имен. Эта ситуация называется коллизией. Хорошая хеш-функция распределяет адреса равномерно, так что коллизии возникают не слишком часто. Но прежде, чем говорить о реализации хеш-функций, обратимся к вопросу о том, каким образом разрешаются коллизии.

46. Особенности интерфейса СУБД.

Для большинства функций СУБД существуют некоторые аналоги на уровне файловых систем. А вот наличие нескольких способов работы с данными является принципиальным отличием СУБД от файлов. Выбор той или иной СУБД на практике часто обуславливается именно удобством работы с ней, а не только производительностью, надёжностью и совершенством архитектуры.

Прямой доступ

Аналогом доступа к файлам можно считать т.н. прямой доступ к СУБД – либо через дополнительные библиотеки того языка, на котором написана

СУБД, либо через специальный язык программирования. Большим преимуществом прямого доступа, безусловно, является высокая производительность. Но так как собственные языки или библиотеки довольно сложны и пригодны для работы лишь с одной СУБД, обычно прямой доступ используют лишь сами разработчики этой СУБД для написания законченных приложений – “дизайнеров”, “навигаторов”, “менеджеров” и других графических оболочек БД. К сожалению, такие приложения можно рассматривать лишь как средства отладки, они неудобны для конечных пользователей, поскольку они не отражают специфику их данных.

Языки баз данных

В связи с этим понятно, почему любая СУБД должна поддерживать хотя бы один универсальный интерфейс, через который можно обращаться к ней из приложений, написанных другими программистами. Ближе всего к прямому доступу находятся интерфейсы, называемые языками баз данных. Их операторы компилируются в исполняемый код специфического для каждой СУБД языка. Языки баз данных раньше делились на языки определения данных и языки манипулирования данными, однако сейчас обе категории операций с БД объединены в интегрированных языках типа SQL. Эти языки стандартизованы, хотя есть различия в реализации языков разными СУБД, и поэтому пока очень редко бывает, чтобы приложение, работавшее с одной СУБД, без изменений могло работать с другой. Инструкции языков баз данных можно последовательно исполнять в специальных программах типа Oracle SQL Plus или Microsoft Quer.

Интерфейсы уровня обращения и их производные

Однако чаще всего языки баз данных используются в так называемых интерфейсах уровня обращения. Для современных языков программирования существуют библиотеки, которые позволяют обращаться через эти интерфейсы к СУБД, пересылая туда инструкции языков баз данных и получая результаты некоторых запросов в виде специальных структур языка программирования. Производители СУБД поставляют свои библиотеки CLI, оптимизированные для их продукта, однако в последнее время наметилась тенденция к стандартизации этих библиотек. В мире реляционных СУБД

наиболее распространены следующие стандартные протоколы программного доступа к БД: ODBC от Microsoft и JDBC от Sun, а также OCI от Oracle. Если программисты используют стандартный интерфейс уровня обращения и придерживаются стандарта языка баз данных, то их приложение сможет работать с разными СУБД. В некоторых средствах программирования существуют также библиотеки, основанные на интерфейсах уровня обращения, которые существенно проще в использовании по сравнению с этими интерфейсами. Примерами могут служить компоненты DAO от Microsoft, Data Express от Borland. При использовании таких библиотек почти не требуется знания языков баз данных. Однако в качестве платы за такое упрощение программист вынужден ограничиться небольшим набором СУБД, которые поддерживаются поставщиками библиотек. Ещё сильнее зависит от конкретной СУБД другой способ программного доступа к БД – встраиваемые в код языка программирования инструкции языка баз данных. Собственные языки программирования, имеющиеся для некоторых СУБД, всегда снабжены синтаксисом встраивания, а производители других СУБД иногда поставляют трансляторы для исходного кода существующих языков, которые заменяют встроенные инструкции языка баз данных на вызовы процедур из специальных библиотек. В обоих случаях набор операций, которые необходимо производить с данными, определяется на этапе компиляции программы, и поэтому рассматриваемый способ доступа к данным называется статическим. Трансляторы встроенных инструкций проверяют их синтаксис и наличие указанных в них логических объектов в конкретной базе данных, а также соответствие между типами данных языка программирования и языка базы данных. Объектные интерфейсы.

Выше речь шла, прежде всего, о реляционных интерфейсах доступа к БД. Во многих современных СУБД поддерживаются также объектные интерфейсы доступа, которые пока имеют менее сложившийся стандарт, чем интерфейсы уровня обращения. Поэтому многие объектно-реляционные СУБД имеют собственные библиотеки, которые обеспечивают автоматическую загрузку и сохранение программных объектов в БД, и лишь немногие СУБД полностью поддерживают стандарт ODMG. Объектный

доступ отличается от реляционного тем, что структуры данных, которыми обменивается программа с СУБД, менее формализованы и фактически совпадают со структурами данных этой программы; поэтому не приходится проводить сложные преобразования результатов каждого запроса к СУБД. Другими словами, работать с хранимыми в базе данных объектами почти так же легко, как с нехранимыми структурами данных языка программирования; необходимо лишь наследовать классы этих объектов от классов написанной другими программистами библиотеки и вызывать некоторые их методы. Встраивание объектных библиотек в приложения может осуществляться не только вручную, но и с помощью утилит, которые устанавливают соответствие между структурами базы данных и структурами объектно-ориентированного языка программирования – классами. Это соответствие может быть как прямым, так и обратным. Обычно утилиты такого типа поставляются разработчиками СУБД; примером “двунаправленной” и независимой от СУБД утилиты является продукт Sun JavaBlend, который преобразует объекты Java в объекты БД и обратно через реляционный интерфейс JDBC. В случае использования объектов абстрагироваться от конкретной СУБД позволяет программное обеспечение промежуточного уровня. Оно избавляет программистов от необходимости знания языков баз данных, интерфейсов уровня обращения или специфических для конкретных СУБД объектных библиотек. При этом разные объекты могут даже храниться в различных СУБД, что обеспечивает программа промежуточного уровня, а для разработчиков конечного приложения важна лишь возможность загрузки и сохранения этих объектов. Достоинством трёхуровневой архитектуры по сравнению с архитектурой “клиент-сервер” является возможность более быстрого создания и развития приложений, а к недостаткам – низкая производительность. К промежуточному уровню относятся программы, реализующие распределённые объектные интерфейсы – DCOM, CORBA, Enterprise JavaBeans. Соотношение реляционных и объектных интерфейсов. Как можно заметить из вышеизложенного, доступ к реляционным БД лучше всего поддаётся стандартизации на низком уровне – на уровне языков баз данных, а на высоком уровне middleware крайне затруднителен. И наоборот,

доступ к объектным БД на уровне языка базы данных плохо стандартизован (пока на роль стандарта претендуют два языка – SQL3 и OQL. Зато использование высокоуровневых распределённых объектных интерфейсов делает приложение совершенно независимым от СУБД. Вообще, роль реляционных и объектных интерфейсов в современных СУБД весьма разнообразна. Исторически сложилось так, что самые уважаемые СУБД имеют реляционное ядро, которое впоследствии дополнилось объектными надстройками. С другой стороны, относительно “молодые” СУБД имеют объектное ядро, по возможности оснащённое реляционным интерфейсом, а постреляционные СУБД, вообще, основана на т.н. многомерной модели данных, по отношению к которой и объектный, и реляционный интерфейсы являются внешними. Естественно, надстройки часто понижают производительность СУБД, что следует учитывать при выборе оптимальной комбинации “СУБД – интерфейс доступа – язык программирования”. Другими словами, с объектной СУБД быстрее работать через объектный интерфейс, с реляционной – через реляционный интерфейс, хотя в случае многомерной СУБД, ориентированной на OLAP-приложения, производительность не “родных” для неё объектных или реляционных запросов на выборку данных тоже достаточно высока. Web-интерфейс. Какими бы удобными для программистов ни были объектные интерфейсы доступа, они требуют некоторых усилий по представлению хранимых в БД объектов пользователю. В частности, при написании Internet-приложений программист отвечает за преобразование получаемых из СУБД результатов запросов и размещение их в графических компонентах апплета; при этом также велика нагрузка на сеть. Использование вместо архитектуры “клиент-сервер” трёхуровневой архитектуры этих проблем не решает. Задача сильно упрощается, если СУБД возвращает результаты запросов к ней вместе с атрибутами их визуального представления, а программисту остаётся лишь задать правила для формирования этих атрибутов. На практике эти результаты отображаются Web-браузером в виде страницы на одном из языков разметки, что даёт дополнительные преимущества: СУБД выходит за рамки локальной сети в глобальную сеть, а пользователи, даже если они находятся в локальной сети с

сервером СУБД, работают в привычных для них программах MS Internet Explorer или Netscape Navigator. Как правило, за преобразование данных из формата СУБД отвечают специальные программы, написанные на скриптовых языках и реализующие интерфейс CGI. Эти программы взаимодействуют с Web-сервером, получая через него запросы Web-броузера и возвращая обратно данные из СУБД. Сетевой трафик при этом меньше, чем при прямой связи клиентского апплета с СУБД. Однако использование внешнего по отношению к СУБД CGI-скрипта – дополнительного звена в и без того длинной цепочке передачи данных – плохо сказывается на производительности, поэтому желательно, чтобы СУБД имела свой Web-интерфейс. При этом, правда, теряется стандартность Web-доступа: каждая такая СУБД будет иметь свой язык скриптов, не соответствующий CGI.